KHUDA

Computer Vision

1주차

머신러닝 개요 및 딥러닝 기초

트랙장 : 이승준

Contents

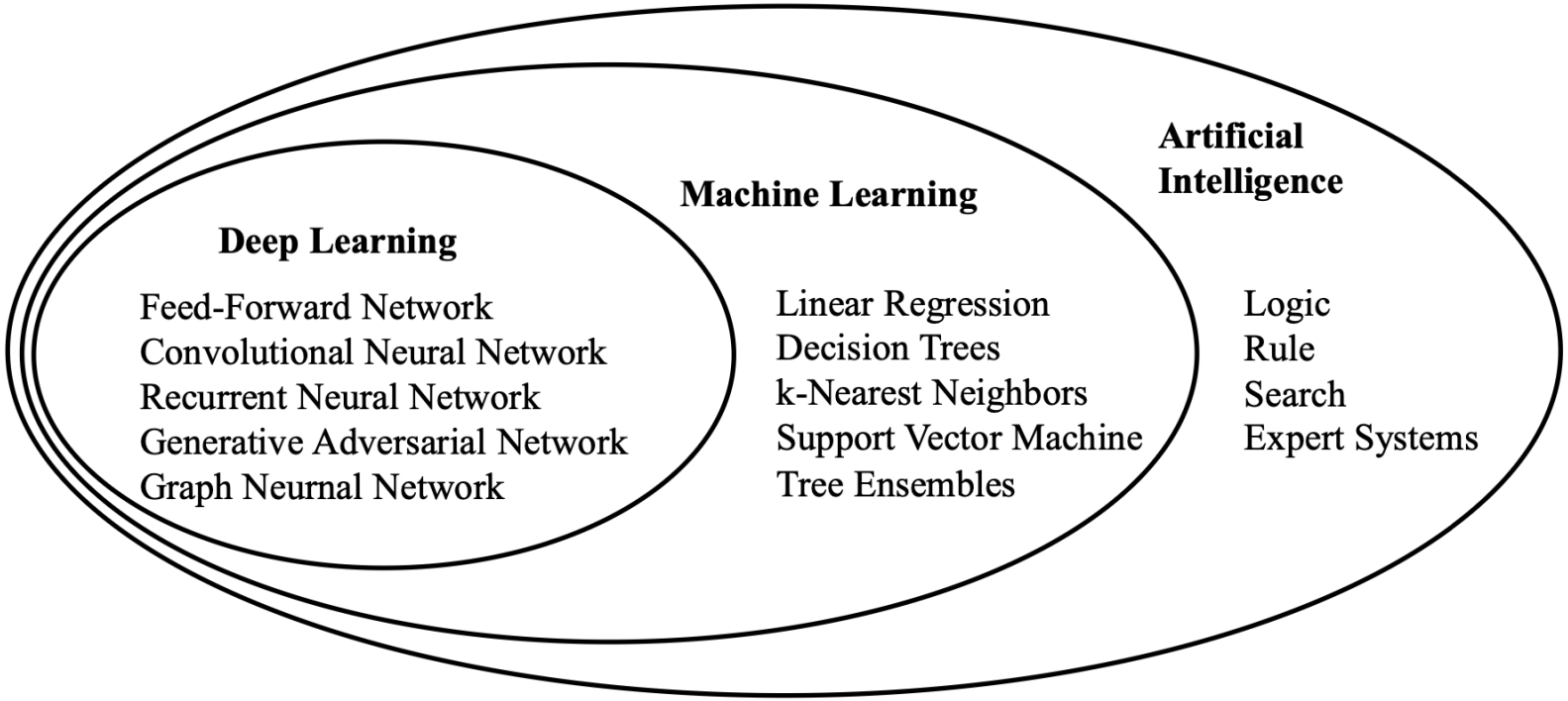
01 머신러닝 개요

02 퍼셉트론 / MLP

03 Gradient Descent

머신러닝 개요

머신러닝, 딥러닝 관계

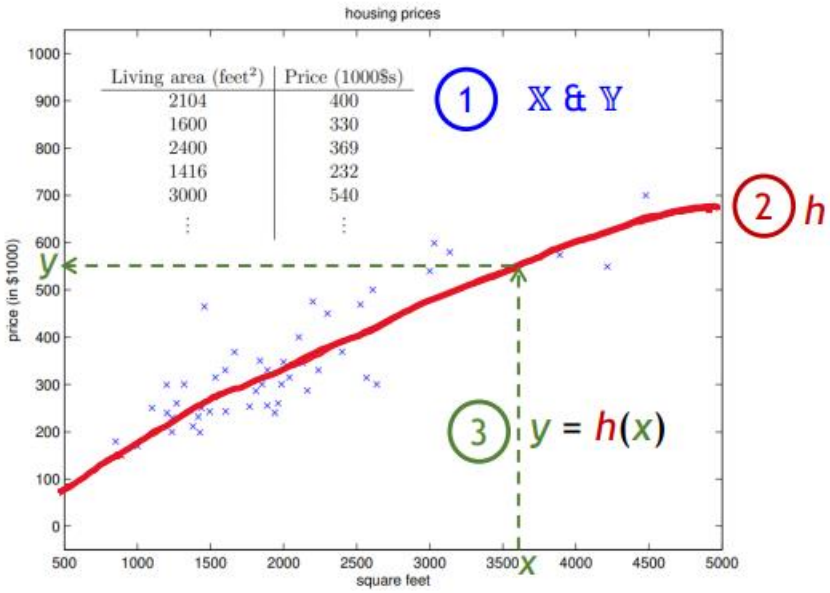

Artificial Intelligence
Logic
Rule
Search
Expert Systems
Machine Learning
Linear Regression
Decision Trees
k-Nearest Neighbors
Support Vector Machine
Tree Ensembles
Deep Learning
Feed-Forward Network
Convolutional Neural Network
Recurrent Neural Network
Generative Adversarial Network
Graph Neurnal Network

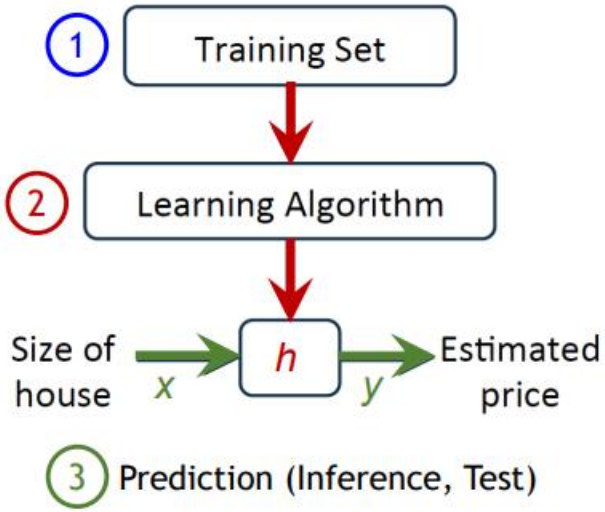
1. Problem Definition

Living area (feet^2)	Price (1000$s)
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮

위와 같은 (평수 – 집 값) 데이터셋이 주어짐 ⟶ 이러한 데이터셋으로 어떤 문제를 해결할 수 있을까?

1. Problem Definition





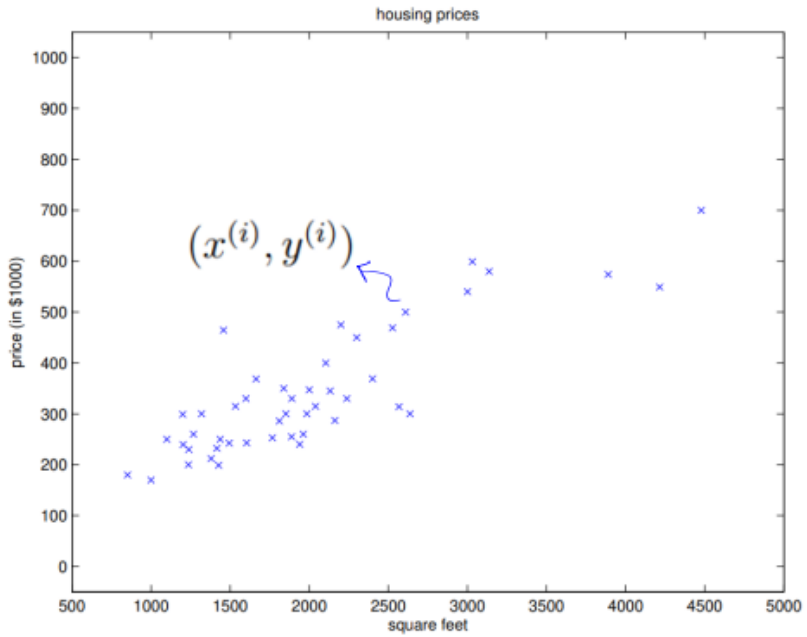
평수를 알면 대략적인 집 값을 알 수 있지 않을까?

⟶ 데이터를 대충 뿌려보니까, 추세가 나오는 것 같은데 저 선을 예측해볼까?

⟶ 어떻게 하면 저 선을 찾을 수 있을까? 어떤 알고리즘을 사용하는게 좋을까?

2. Data Representation

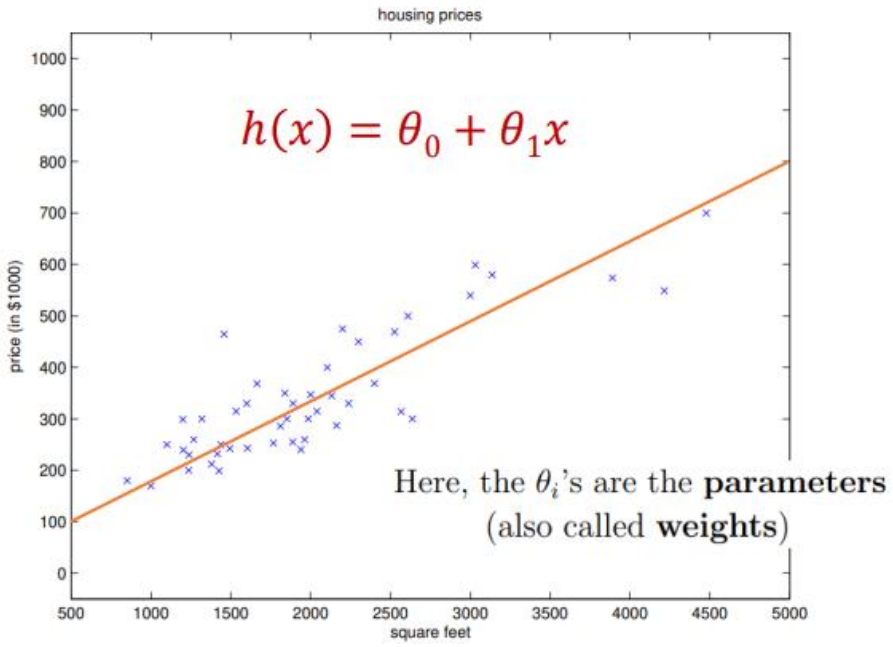
Index	x Living area (feet^2)	y Price (1000$s)
1	2104	400
2	1600	330
3	2400	369
4	1416	232
5	3000	540
	⋮	⋮
n	2600	400



Input / Output은 무엇이지?

이런 데이터 구조에 적합한 모델은 무엇일까?

3. Model Representation

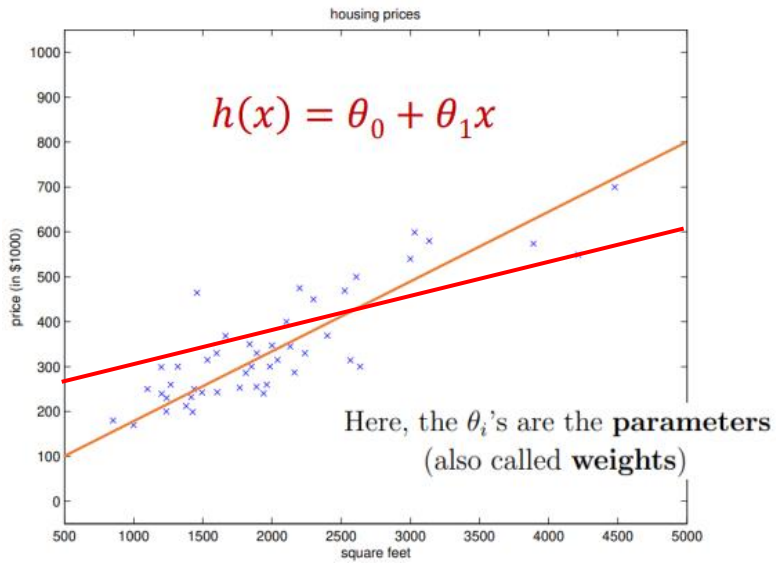


모델을 찾는다는 것은 무엇을 뜻하는거지?

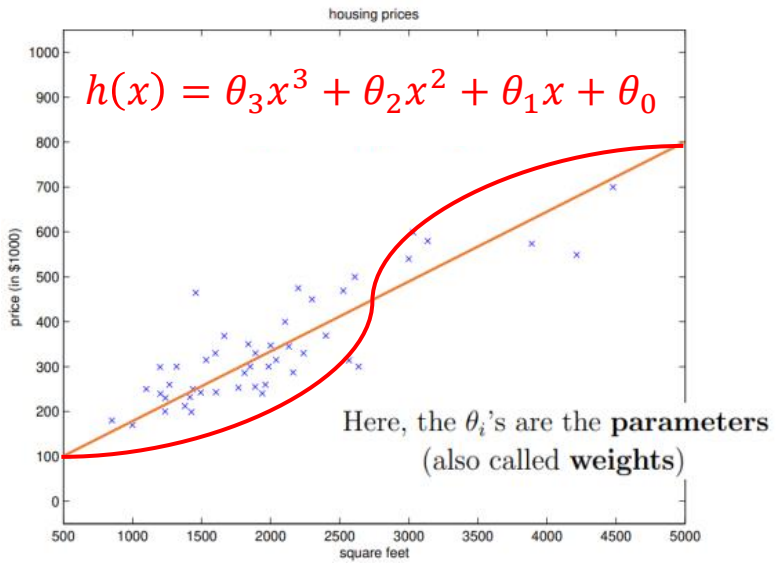
1. 데이터를 가장 잘 설명할 수 있는 모델 구조를 찾는다
2. 특정 모델 구조 속에서 최적의 Parameter를 찾는다

3. Model Representation

Model A



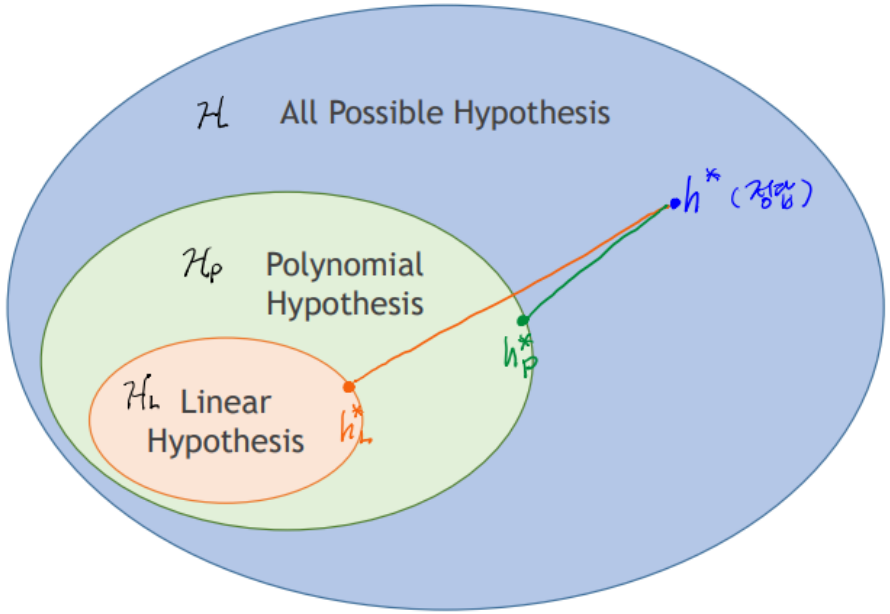
Model B



1. 데이터를 가장 잘 설명할 수 있는 모델 구조를 찾는다 -> Regression Model

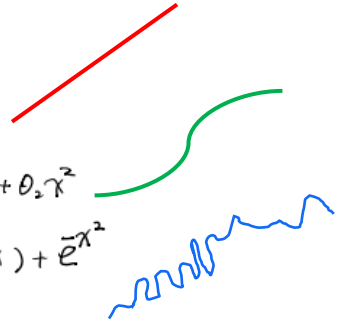
다양한 Regression 모델의 형태 중 어떤 선을 찾아야 하는 것이지?

3. Model Representation



Examples

- $h \in \mathcal{H}_L$: $h(\underline{x}) = \theta_0 + \theta_1 x$
- $h \in \mathcal{H}_P$: $h(\underline{x}) = \theta_0 + \theta_1 x + \theta_2 x^2$
- $h \in \mathcal{H}$: $h(x) = \theta_0 + \cos(x) + e^{-x^2}$

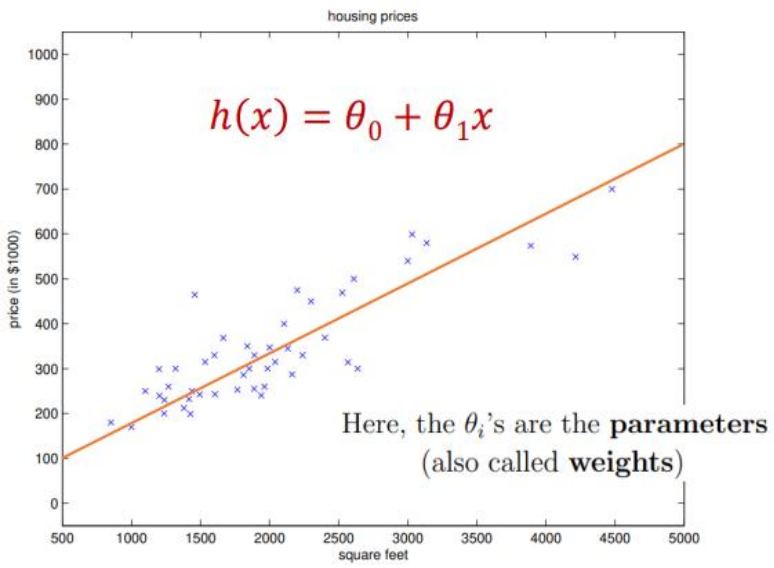


다양한 Regression 모델의 형태 중 어떤 선을 찾아야 하는 것이지?

모든 데이터를 다 설명할 수 있는 고차원 함수를 찾을 수도 있고,
적당한 제약이 있는 상황 속에서는 1차 함수를 택할 수도 있는 것임 ⟶ 우리가 세운 가설 속에서, 가장 정답에 가까운 녀석을 찾아보자

3. Model Representation

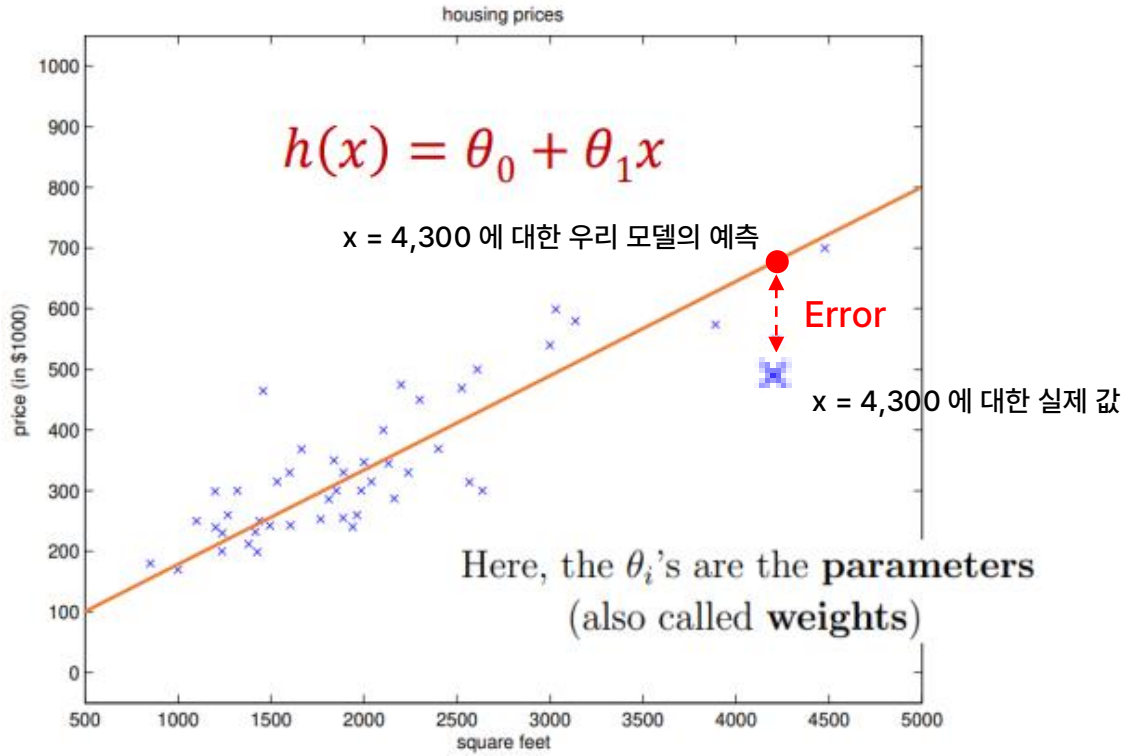
Final Model

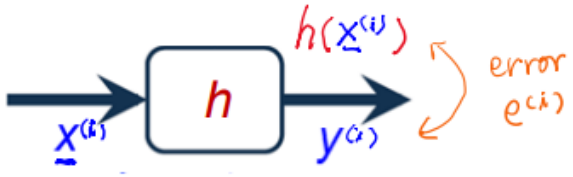


2. 특정 모델 구조 속에서 최적의 Parameter(θ_0, θ_1)를 찾는다

최적의 Parameter(θ)는 어떻게 찾는거지 ? ⟶ 정답을 찾아주는 녀석이 바로 Cost Function, Optimizer 등

4. Cost Function

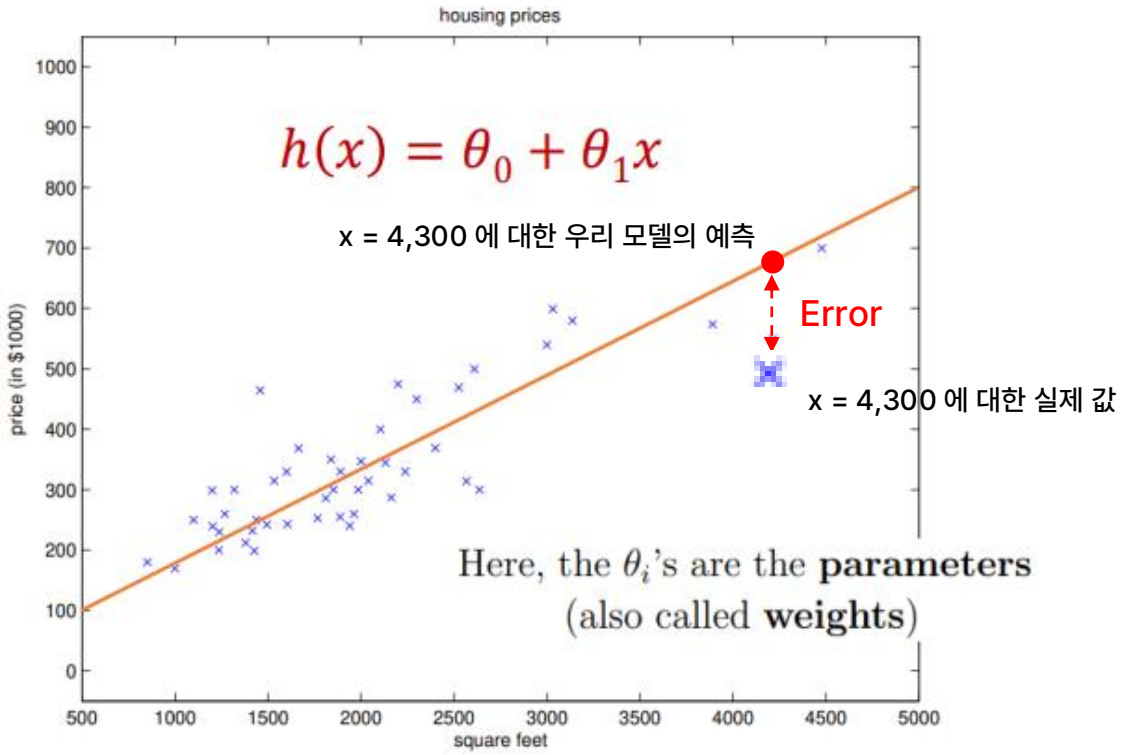




Cost Function

모델의 예측이 정답에서 멀어질수록 Penalty를 부여하는 녀석

4. Cost Function



1번 데이터의 error = (-) 300 달러
2번 데이터의 error = (+) 300 달러
...
N번 데이터의 error = (+) 30 달러

⟶ 전체 Error는 +30 달러 ??

Ex) Minimize MSE Cost (Loss):

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\underbrace{(h_\theta(x^{(i)}) - y^{(i)})}_{\triangleq\ e^{(i)}}{}^2$$

"지금 모델이 너무 값을 뻥튀기해서 예측하는 것 같은데, 300을 조금 줄일까?"

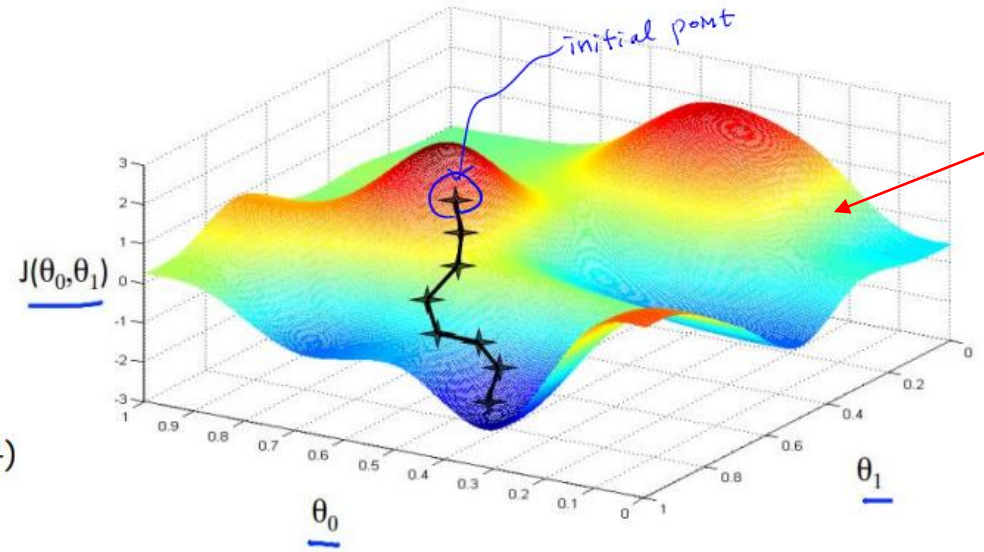
"근데 얼마만큼 줄여야 모델이 더 좋아지는거지..?"

Y = 10x + 300 ⟶ Y = 10x + 290

모델에서 측정한 Error를 통해서 Parameter를 어떻게 수정해야하지?

5. Optimization

- **Iterative Methods (반복법)**
  - Gradient Descent (GD)
  - Stochastic Gradient Descent (SGD)
  - Newton's Method
- **Analytic Methods (해석적 방법)**
  - Normal Equation (Linear Model의 경우)



$$J(\theta) = \frac{1}{2}\sum_{i=1}^{n}\underbrace{(h_\theta(x^{(i)}) - y^{(i)})}_{\triangleq\, e^{(i)}}{}^2$$

<u>Gradient Descent</u>

Parameter를 조금씩 이동하면서, $J(\theta)$를 최소화 하는 방향으로 이동

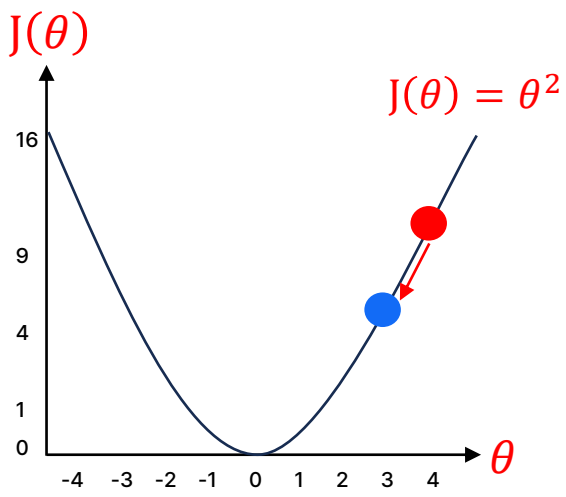
<u>Normal Equation</u>

$J(\theta)$를 미분한 값이 0이 되도록 하는 Parameter값 찾기

5. Optimization

<u>Gradient Descent 맛보기</u>

Parameter를 조금씩 이동하면서, $J(\theta)$를 최소화 하는 방향으로 이동



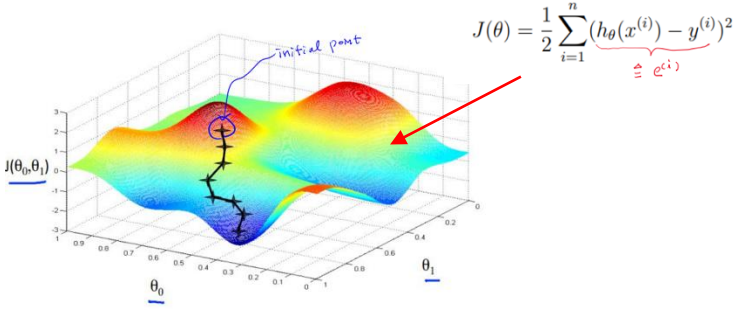
1. 비용함수가 감소하는 방향을 찾아낸다

함수의 미분인 기울기는 증가하는 방향이니까, 기울기의 반대방향은 감소하는 방향

$$\nabla J(\theta) = J'(\theta) = 2\theta$$

2. 찾아낸 방향으로 Parameter를 갱신한다 $\alpha = learning\ rate\ (Hyperparameter)$

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$



Example) 학습률은 0.1이라 가정

현재 θ가 4일때, $J(\theta)$가 16임

$new\ \theta = 4 - 0.1 * 2 * 4 = 4 - 0.8 = 3.2$

수정된 new θ는 3.2이고, 이때 비용함수는 $3.2^2 = 10.24$

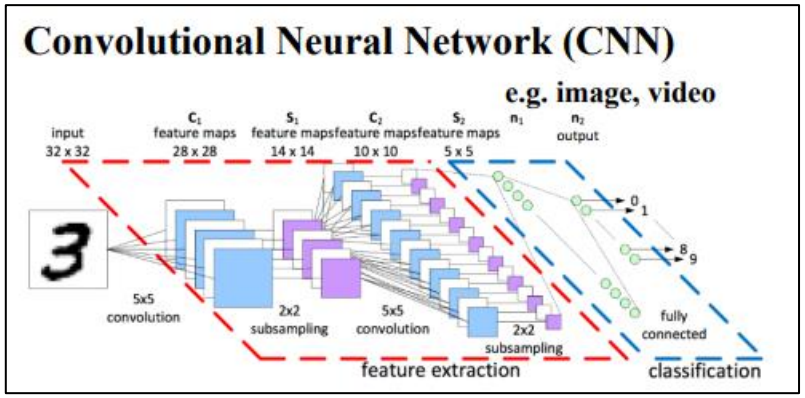
앞으로 배울 것들

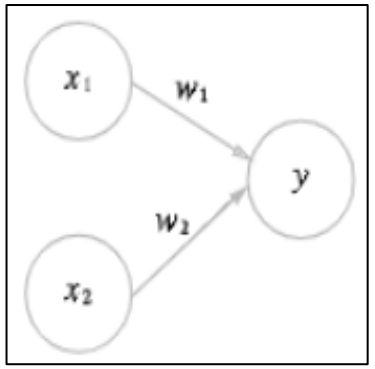
1. 이미지 데이터를 input으로 하는 모델 구조

2. 이미지 데이터를 다루는 다양한 task에 대해 (서로 다른 Output)

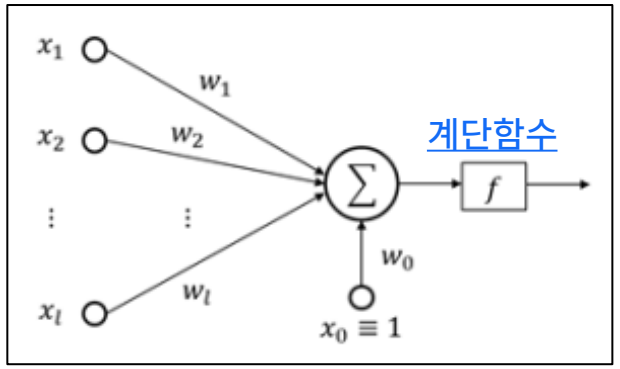
3. 서로 다른 Output에 맞는 Cost Function, Activation Function에 대해

4. 깊은 모델을 다루기 위한 다양한 technique

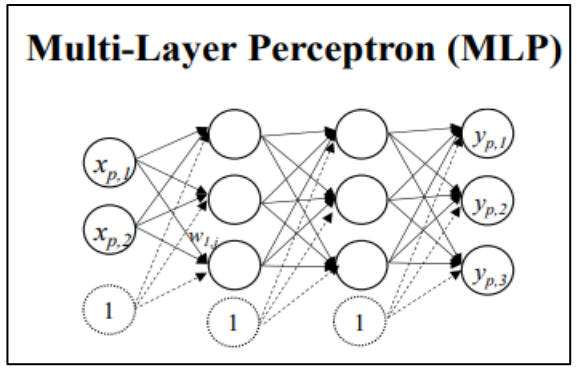




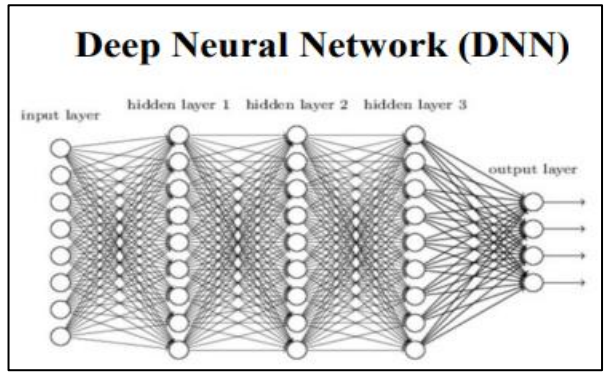
선형 모델



퍼셉트론



다층 퍼셉트론

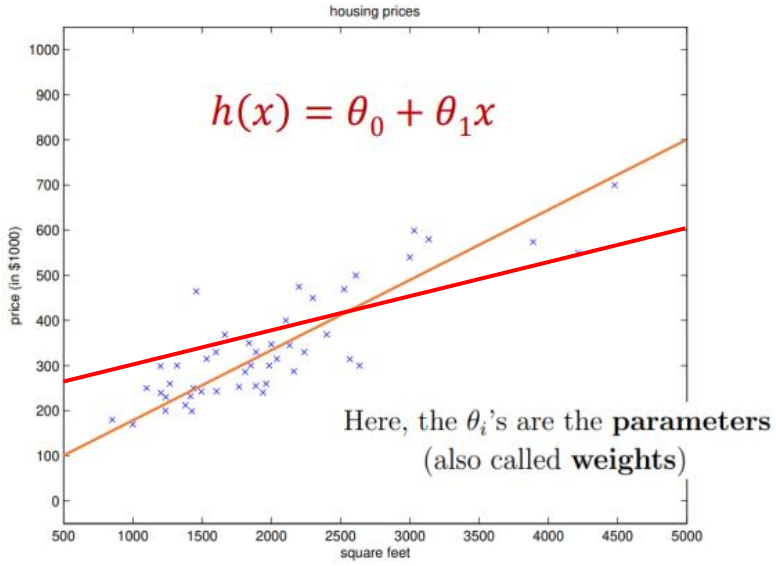


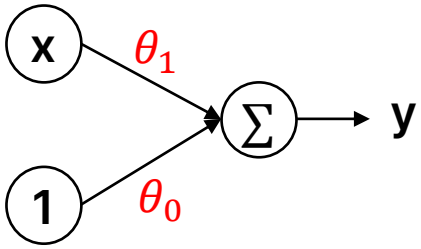
심층 신경망

퍼셉트론

Perceptron

선형 모델

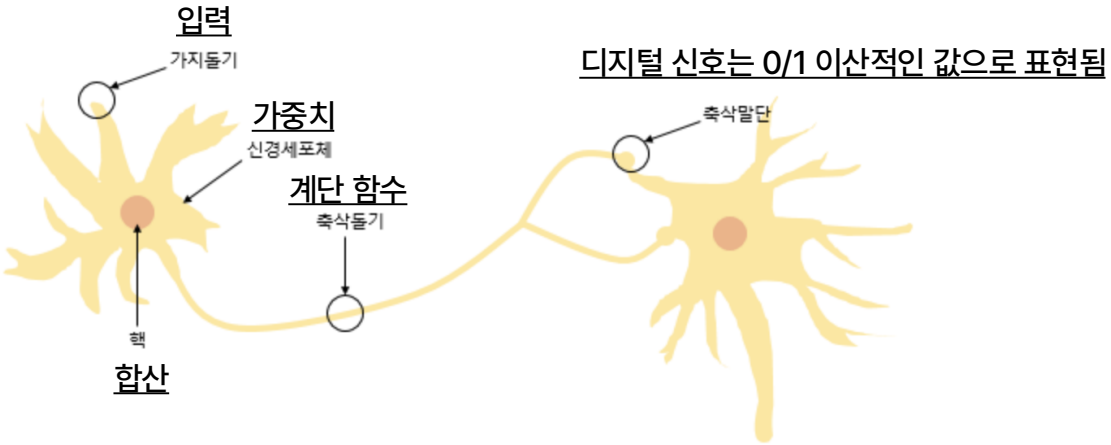


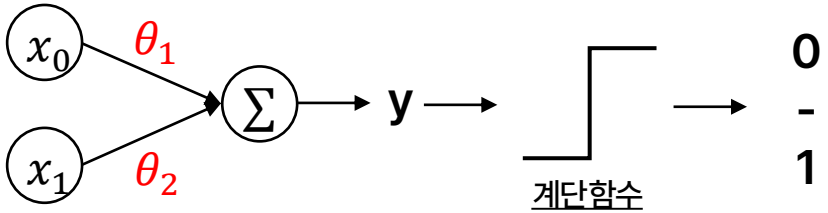


퍼셉트론

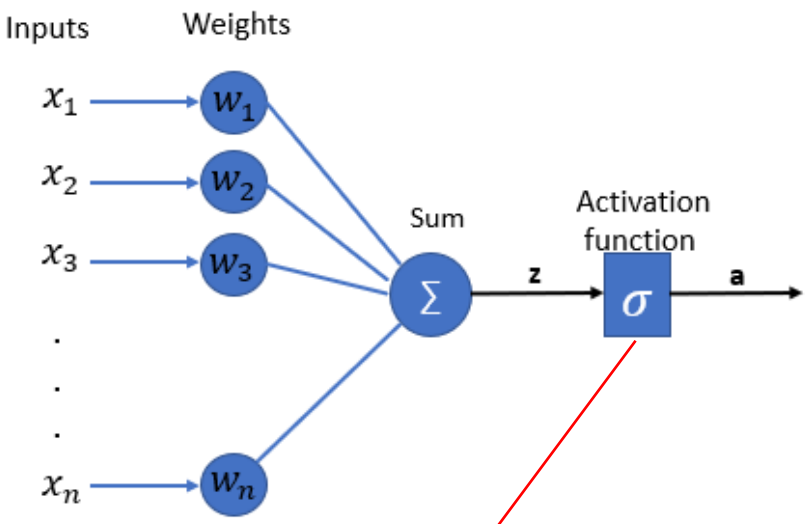
1. 퍼셉트론(Perceptron)

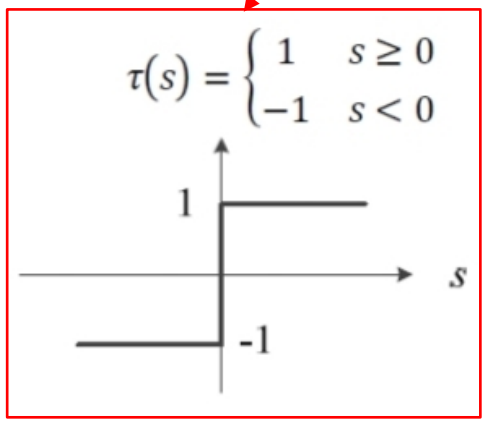
퍼셉트론(Perceptron)은 프랑크 로젠블라트(Frank Rosenblatt)가 1957년에 제안한 초기 형태의 인공 신경망으로 다수의 입력으로부터 하나의 결과를 내보내는 알고리즘입니다. 퍼셉트론은 실제 뇌를 구성하는 신경 세포 뉴런의 동작과 유사한데, 신경 세포 뉴런의 그림을 먼저 보도록 하겠습니다. 뉴런은 가지돌기에서 신호를 받아들이고, 이 신호가 일정치 이상의 크기를 가지면 축삭돌기를 통해서 신호를 전달합니다.





Perceptron 동작 방식





0/1 이산적인 값이 들어왔을 때,
값들과 가중치들의 weighted summation이
threshold(0.5)를 넘으면 1로 출력 / 넘지 못하면 0으로 출력

계단 함수

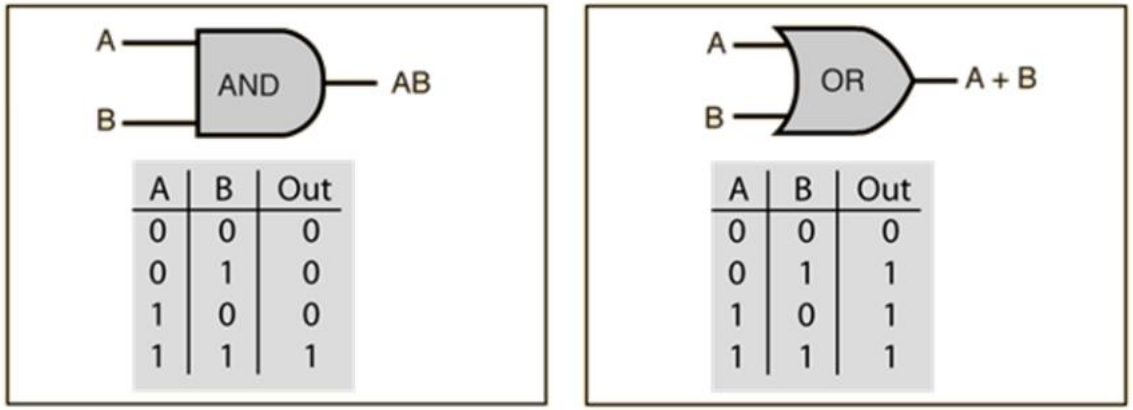
$$Output = \sigma(\underbrace{w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n}_{\text{Input과 weights의 weighted summation}}) = 0/1$$

Perceptron의 동작 방식

$$Output = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n)$$

결국 Perceptron이 찾아내는 것은 weight들이 만들어내는 선형성을 띄는 결정 경계

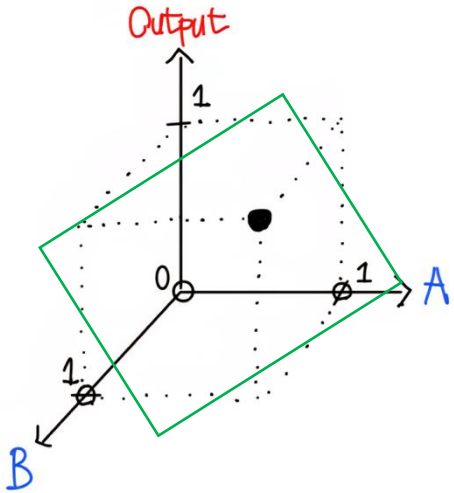
직선(line) -> 평면(plane) -> 초평면(hyperplane)



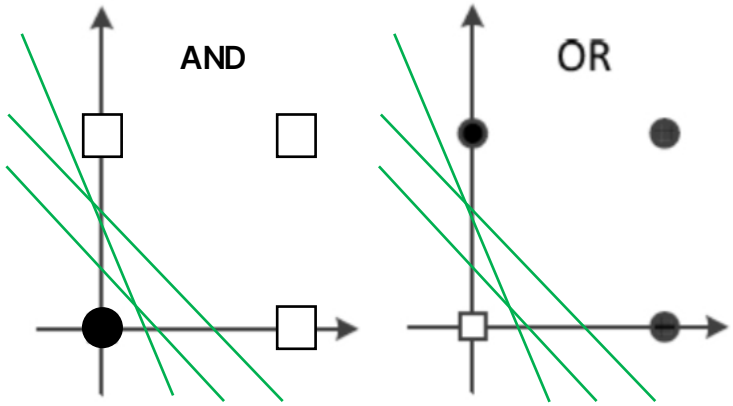
A	B	Out
0	0	0
0	1	0
1	0	0
1	1	1

A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

3차원에 그린 AND Gate



2차원에 그린 AND/OR Gate



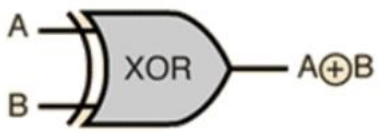
Perceptron의 문제점

$$Output = \sigma(w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n)$$

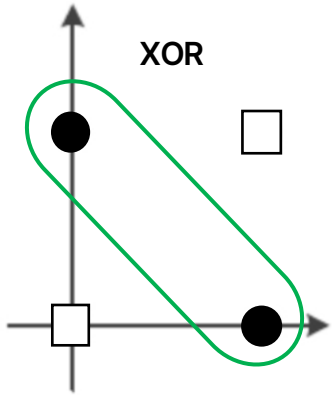
결국 Perceptron이 찾아내는 것은 weight들이 만들어내는 선형성을 띄는 결정 경계

직선(line) -> 평면(plane) -> 초평면(hyperplane)

XOR Gate처럼 선형 분리가 불가능한 상황이 있음

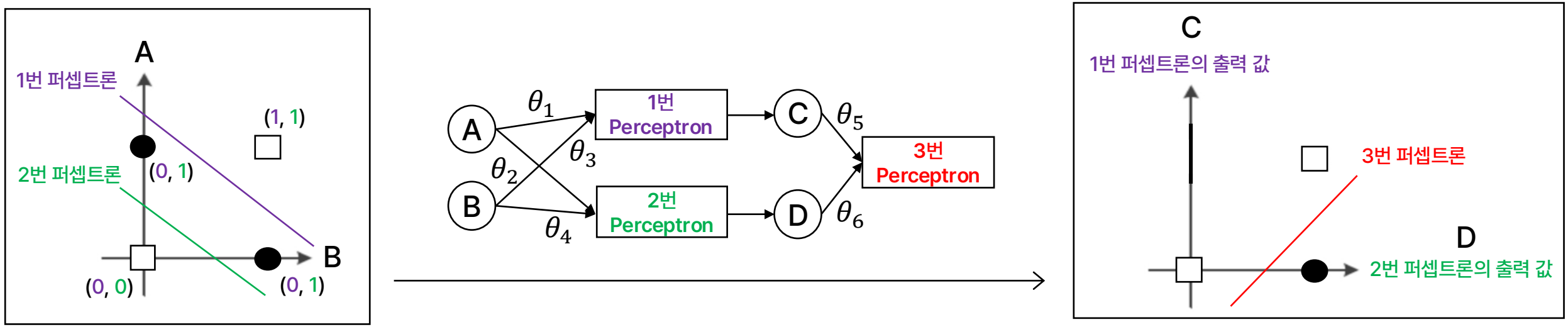


A	B	Out
0	0	0
0	1	1
1	0	1
1	1	0



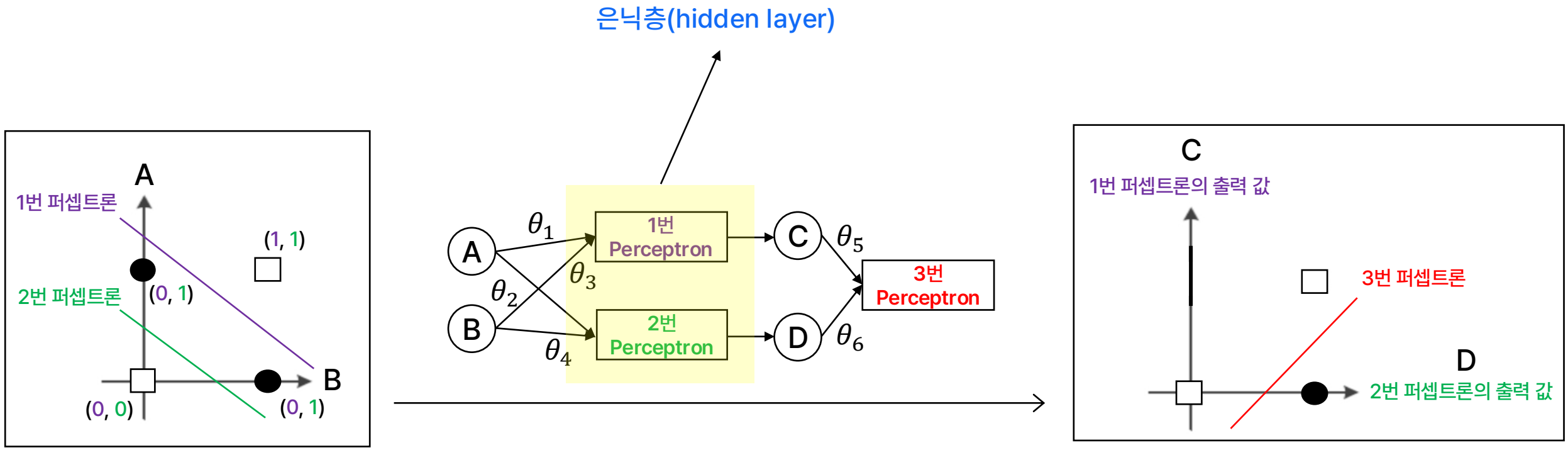
Perceptron의 문제점

XOR Gate처럼 선형 분리가 불가능한 상황이 있음 ⟶ 여러 개의 퍼셉트론을 사용하면 해결 가능함



원래 특징 공간 (A, B)에서 해결할 수 없던 문제를 새로운 특징 공간 (C, D)로 변환함으로써 해결함

Multi-Layer Perceptron



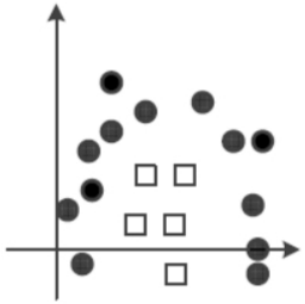
은닉층(hidden layer)의 도입

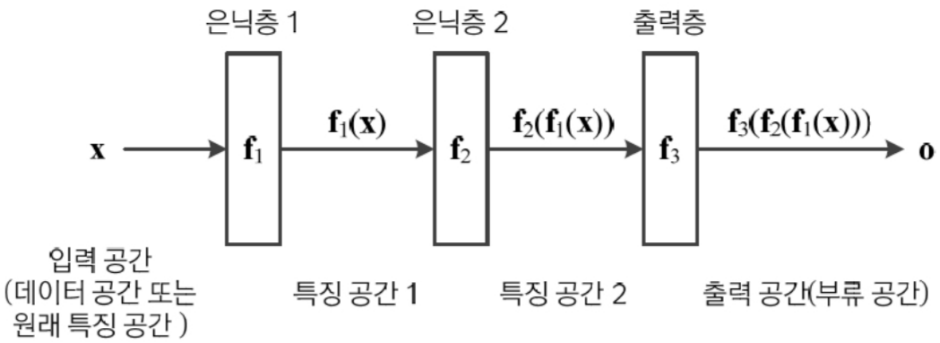
★ 은닉층을 통해 원래 특징 공간을 분류에 유리한 새로운 특징 공간으로 변환 가능

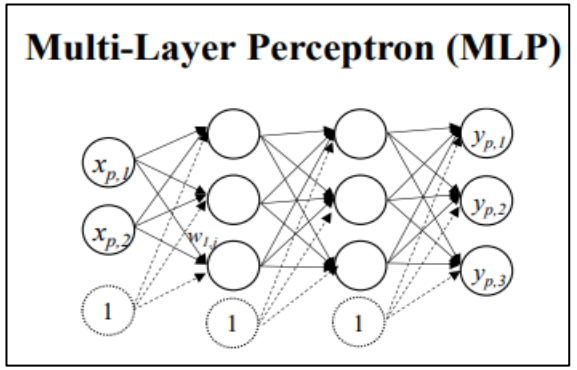
★ 이전 특징공간에서 선형분리 불가능한 문제를 변환된 특징공간에서 선형분리 가능하도록 할 수 있음
(예: XOR 문제의 해결)

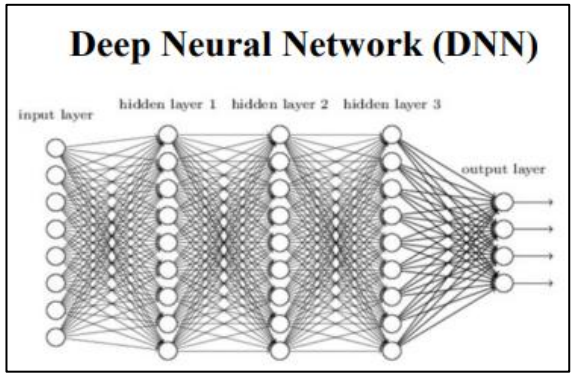
Multi-Layer Perceptron

2차원 복잡한 데이터





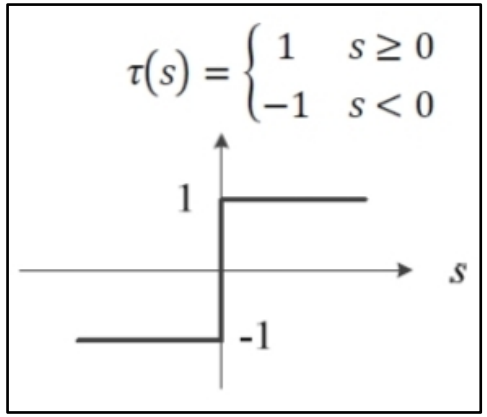


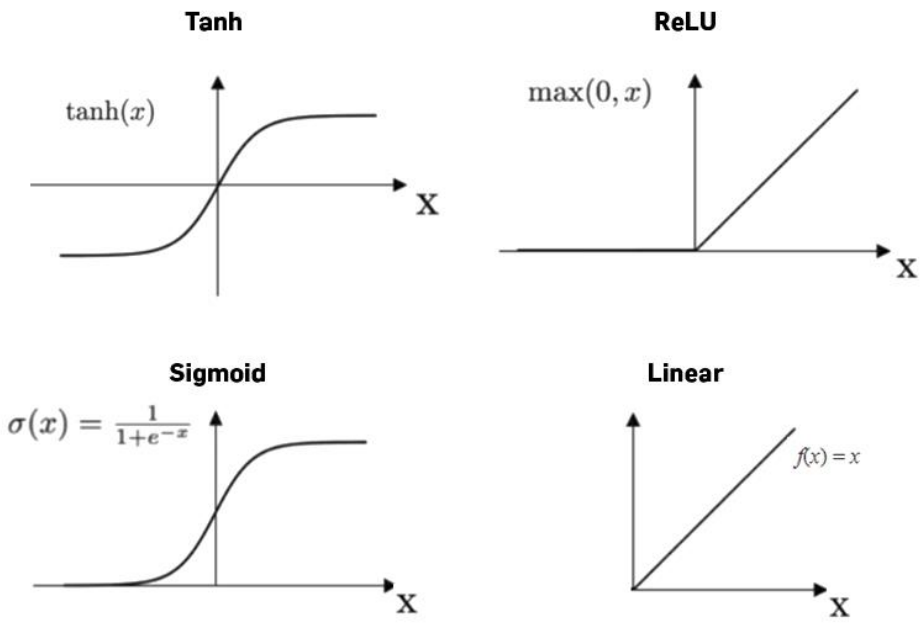


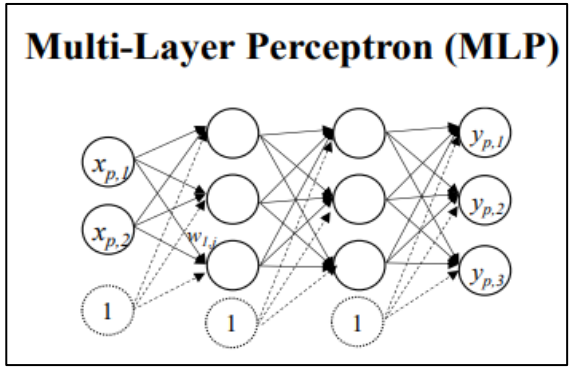
우리가 다루는 데이터는 고작 2차원에서 끝나지 않습니다.
이미지 한 장이 1024x1024 해상도라면, 1024 x 1024개의 픽셀별로, 3개의 channel 값이 있음

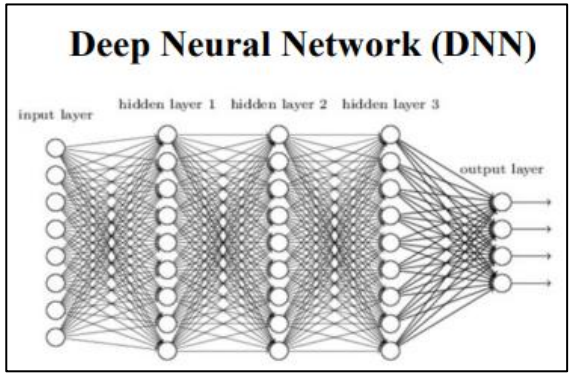
Multi-Layer Perceptron

엥 ? 계단함수를 쓰면 Output은 항상 0/1으로 binary한데, 우리가 원하는 다양한 task는 어떻게 진행하죠?



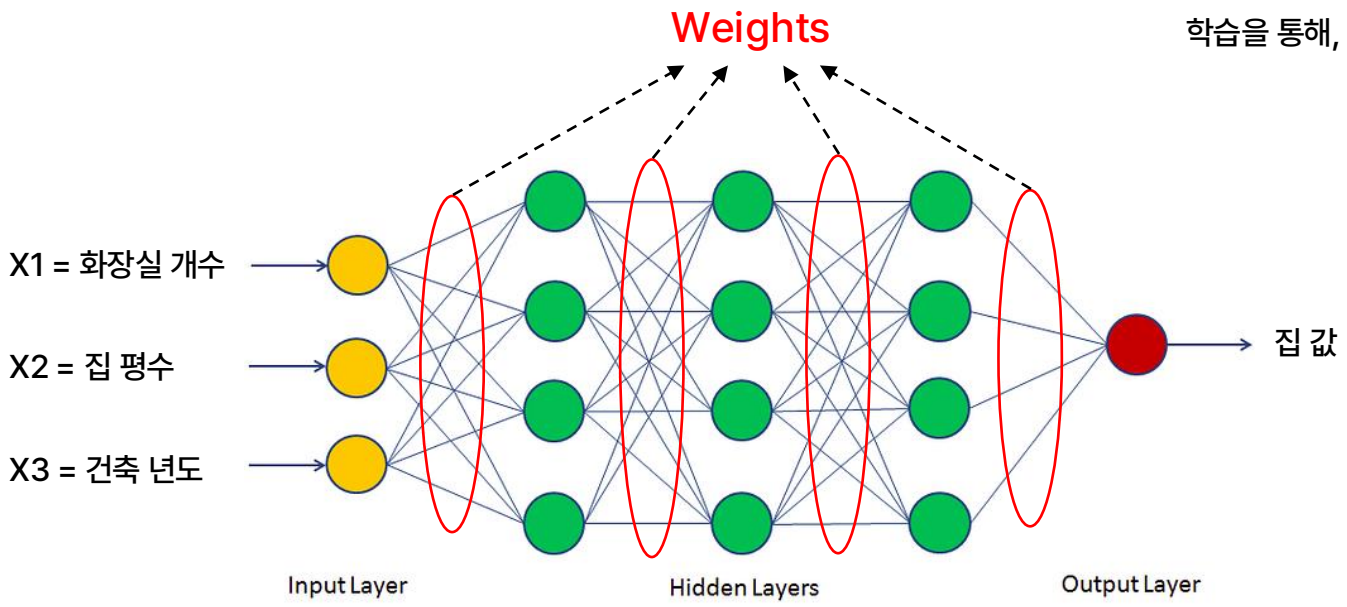






딱딱한 계단 함수를 activation function으로 사용하는 것이 아니라,
연성 활성함수를 사용하여 부드러운 공간 분할이 가능하게 함

Multi-Layer Perceptron



MLP를 훈련시킨다는 것은 무엇이지?

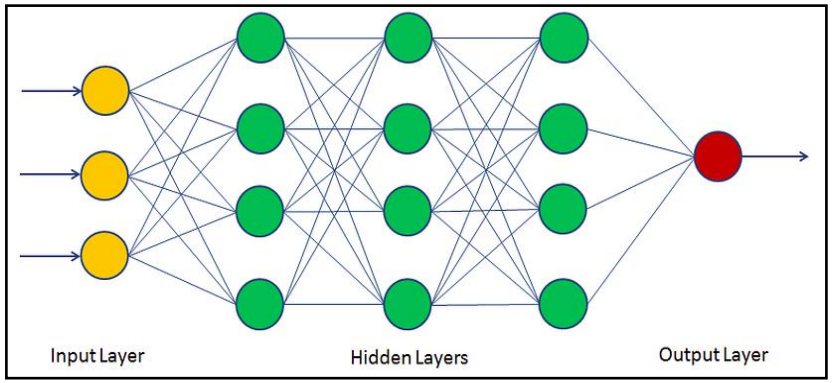
학습을 통해, 우리가 원하는 Output을 만들어주는 Weights를 찾는 것

Ex)
X1 = 화장실 개수, X2 = 집 평수, X3 = 건축 년도

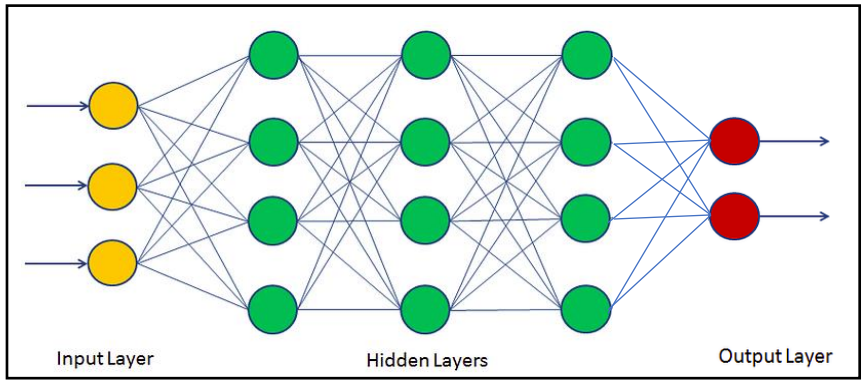
(2, 32, 2020)이라는 Input을 보고, 정답인 5억을 말할 수 있는 Weights를 찾기

Multi-Layer Perceptron

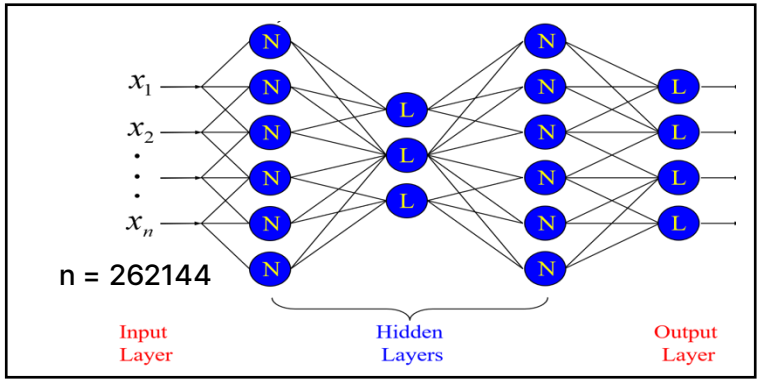
A



B



C



문제 1)

– 평수, 화장실 개수, 건축 년도 3가지 input을 통해 집 값을 예측하는 경우, 어떤 모델 형태를 따라야 하는가?

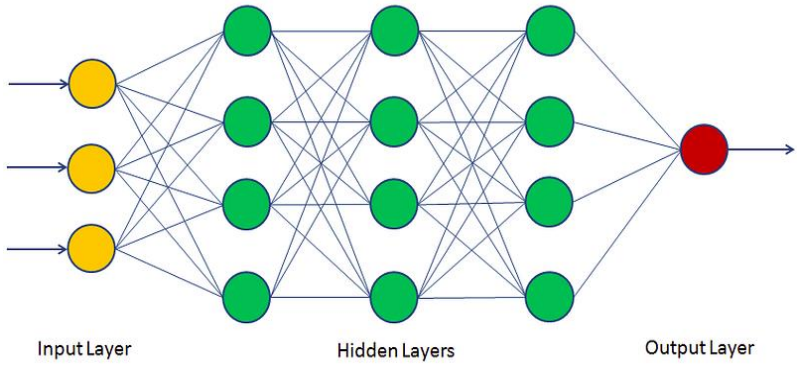
문제 2)

– 하루 칼로리 섭취량, 하루 끼니 수, 부모님 키 3가지 input을 통해 몸무게와 비만율을 예측하는 경우, 어떤 모델 형태를 따라야 하는가?

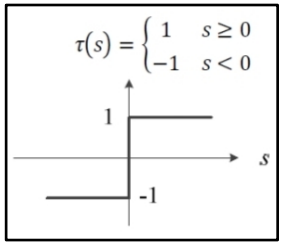
문제 3)

– 해상도가 512x512이며, channel이 1개인 사진 한 장을 통해 사진 속 동물의 bbox를 예측하는 경우, 어떤 모델 형태를 따라야 하는가?

Multi-Layer Perceptron



수치를 예측하는 모델인데,
계단 함수를 사용할 수 없음



아무튼 모델을 학습시키기 위해선 다음과 같은 기본적인 개념을 제대로 숙지해야 함

Activation Function (활성화 함수)
– 내가 원하는 출력을 뽑아내기 위해 적합한 활성화 함수는 무엇인가? Hidden Layer에 어울리는 활성화 함수는 무엇인가?

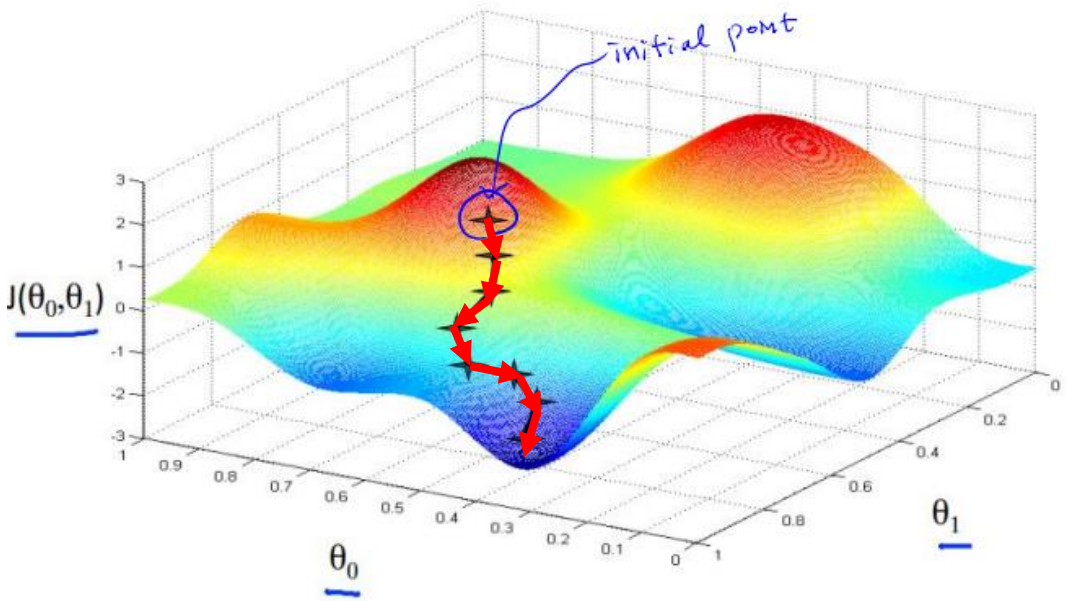
Cost Function
– 내가 원하는 출력을 뽑아내기 위해 어떤 Cost Function으로 모델을 가르칠 것인가? (분류, 회귀 등 output에 따라 서로 다른 cost function을 사용함)

Optimizer
– 방대한 데이터를 학습시키기 위해 적합한 Optimizer는 무엇인가? Gradient Descent는 어떻게 동작하는가?

기타 technique
– Hidden Layer가 깊으면 깊어질수록 가중치 갱신이 어려운데, 해결하는 technique에는 무엇이 있는가?

Training - Gradient Descent

How to Train Model – Gradient Descent



$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

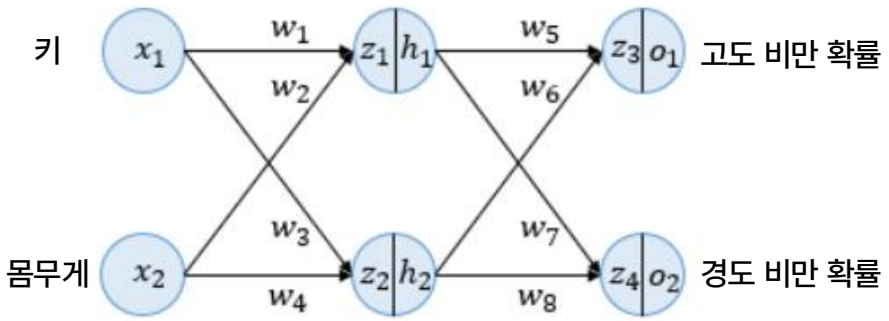
비용 함수라는 산의 정상에서 시작해서
한 step 한 step 거쳐가면서 비용함수를 줄여나가는 방향으로
가중치를 수정하면서 산을 내려오겠다

1. 현재 모델에 특정 데이터를 넣고 Output을 만들어낸다
2. 현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다
3. 계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다
4. Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

간단한 예시로 배우는 Gradient Descent

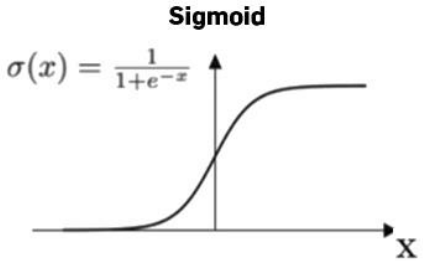
간단한 상황을 예시로 들어 모델 학습 과정을 이해해봅시다.

가장 간단한 Input 2 – Output 2 / 단층 MLP



1. Input : 키 / 몸무게 -> Output : 고도 비만 확률 / 경도 비만 확률
2. h1, h2, o1, o2는 z1, z2, z3, z4가 Sigmoid 함수를 통과한 이후의 수치임
   z1 = w1*x1 + w2*x2 -> h1 = f(z1)

입력 값을 0 ~ 1 사이로 만들어주는 비선형 함수



현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

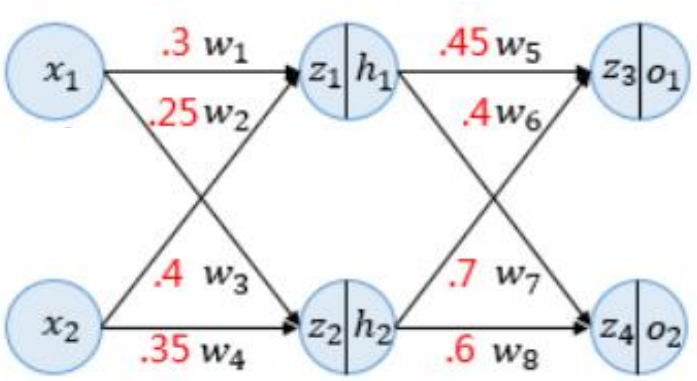
현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

+) Weight Initialization

물론, 더 빠르고 안정적인 학습을 위해 가중치를 초기화하는 technique이 있지만
지금은 그냥 동작을 이해하기 위해 아무 숫자로 무작위 초기화를 진행함



시작 지점이 좋은 weight라면 훨씬 더 안정적이고 빠르게 정답에 가까워질 수 있음
따라서, 그냥 범위도 없이 무작위 초기화는 정말 안 좋은 행동임

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

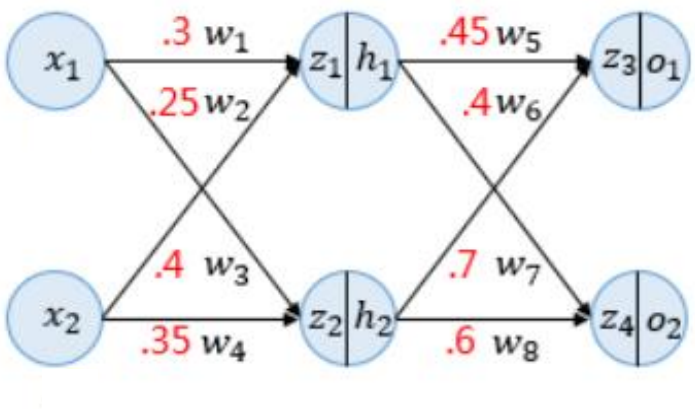
Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

1. Forward Propagation

	키 (x1)	몸무게 (x2)
1번 데이터	0.1	0.2

0.1

0.2



1. z1, z2 계산하기

$$z_1 = w_1x_1 + w_2x_2 = 0.3\times0.1 + 0.25\times0.2 = 0.08$$

$$z_2 = w_3x_1 + w_4x_2 = 0.4\times0.1 + 0.35\times0.2 = 0.11$$

2. h1, h2 계산하기

$$h_1 = sigmoid(z_1) = 0.51998934$$

$$h_2 = sigmoid(z_2) = 0.52747230$$

$$\phi(z) = \frac{1}{1+e^{-z}}$$

3. z3, z4 계산하기

$$z_3 = w_5h_1 + w_6h_2 = 0.45\times h_1 + 0.4\times h_2 = 0.44498412$$

$$z_4 = w_7h_1 + w_8h_2 = 0.7\times h_1 + 0.6\times h_2 = 0.68047592$$

4. o1, o2 계산하기

$$o_1 = sigmoid(z_3) = 0.60944600$$

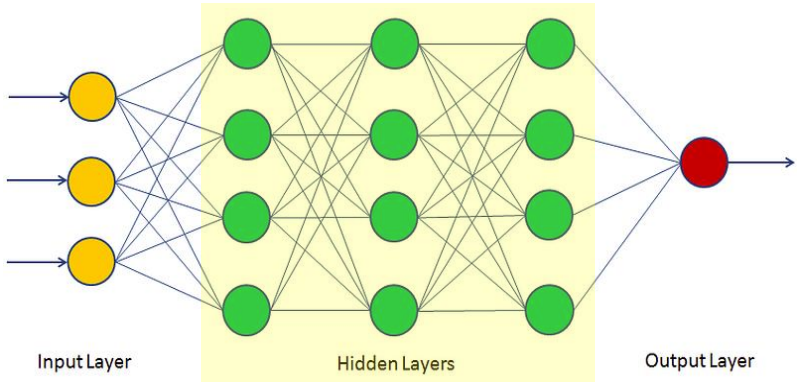
$$o_2 = sigmoid(z_4) = 0.66384491$$

$$\phi(z) = \frac{1}{1+e^{-z}}$$

+) Activation Function

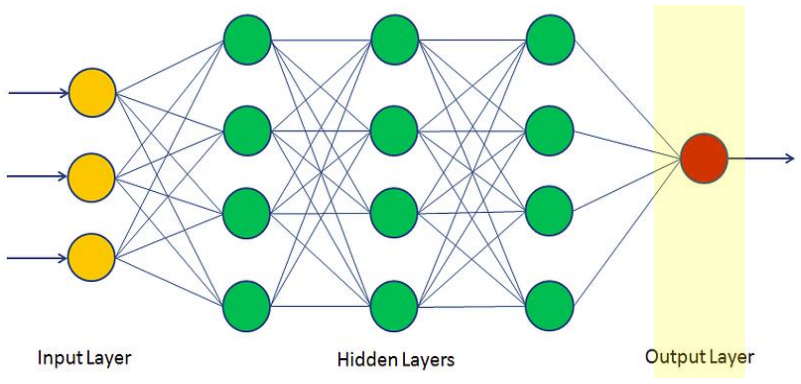
Activation Function이 정확히 무엇이고, 어떤 역할을 수행하는거죠?

Hidden Layer에 속하는 Activation Function의 관점



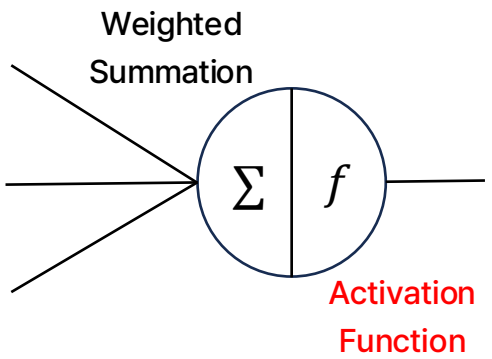
Activation Function은 뉴런의 가중합을 입력으로 받아,
이를 비선형적으로 변환함으로써 신경망이
복잡한 패턴과 비선형 함수를 근사할 수 있게 해주는 함수

Output Layer에 속하는 Activation Function의 관점



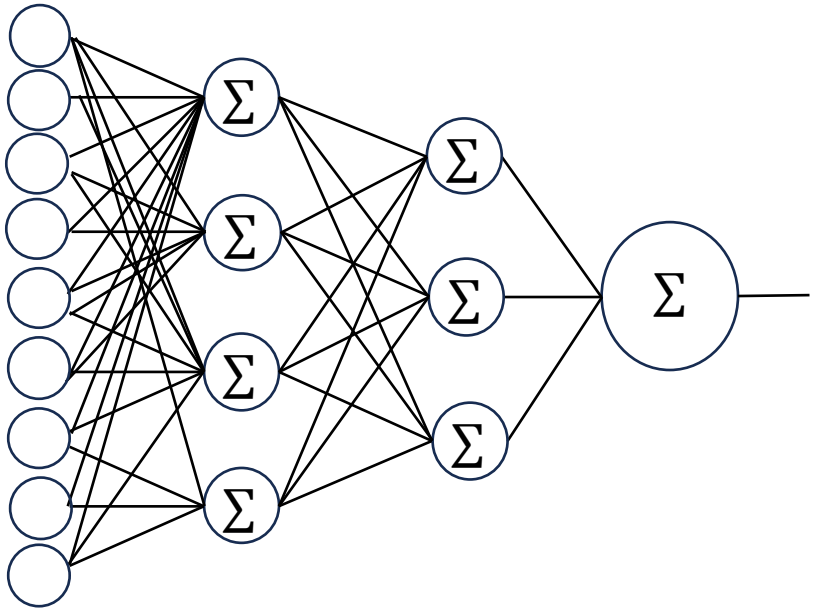
우리가 원하는 Task에 맞는 Output을 뽑아내기 위해
노드에 들어온 값들을 적절히 변환하여 출력하는 역할을 함

+) Activation Function

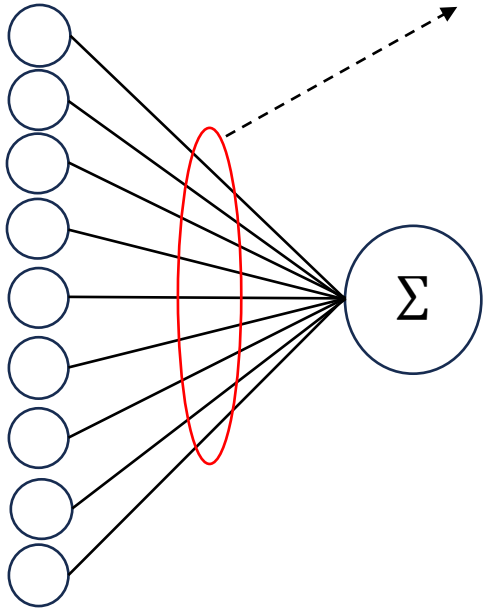


Hidden Layer에 속하는 Activation Function의 관점

Activation Function은 뉴런의 가중합을 입력으로 받아,
이를 비선형적으로 변환함으로써
신경망이 복잡한 패턴과 비선형 함수를 근사할 수 있게 해주는 함수



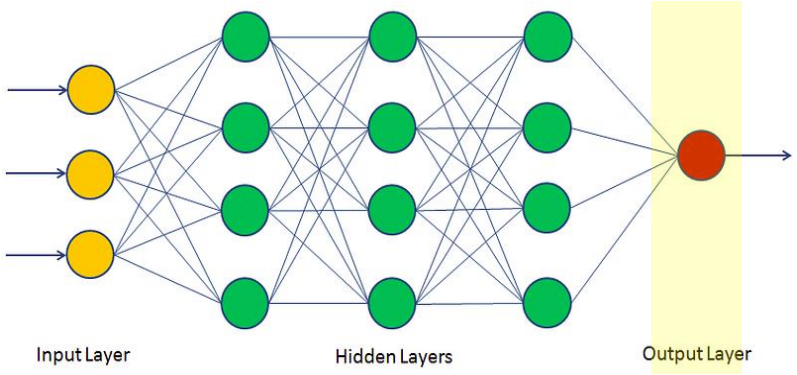
예쁜 가중치 한방에 가능



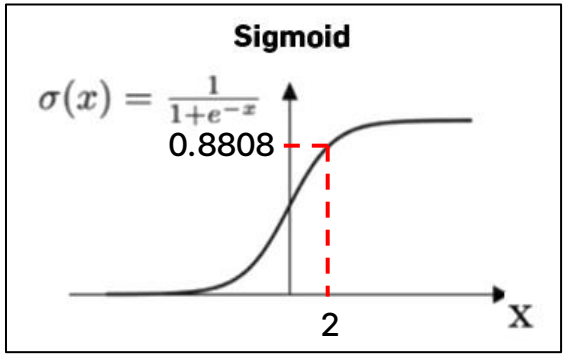
Activation Function이 Linear Function이라면,
모델이 아무리 깊고, 층이 많아도 결국 선형 변환 하나로 단순화할 수 있음
즉, 은닉층을 몇 개 쌓아도 선형 회귀랑 똑같은 모델이 되는 것임

+) Activation Function

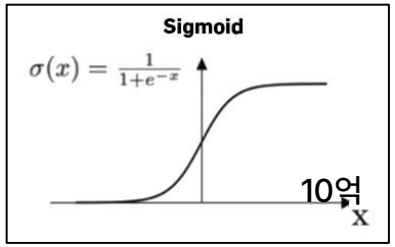
Output Layer에 속하는 Activation Function의 관점은?

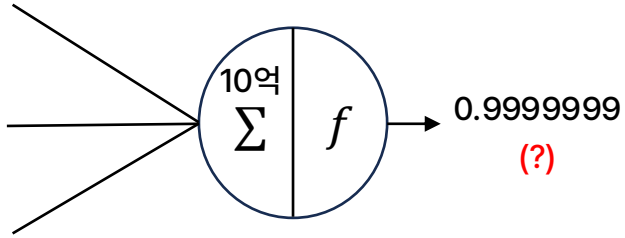


우리가 원하는 Task에 맞는 Output을 뽑아내기 위해
노드에 들어온 값들을 적절히 변환하여 출력하는 역할을 함

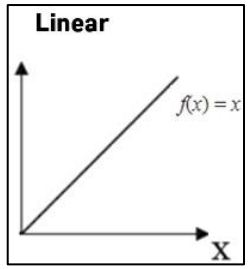


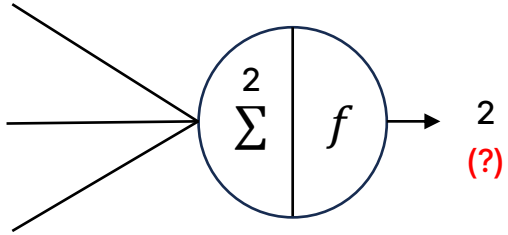
Ex) 집 값 예측 모델인데, output node에 sigmoid를 끼우면?





Ex) 집을 살 수 있는 확률 예측인데, Linear를 끼우면?

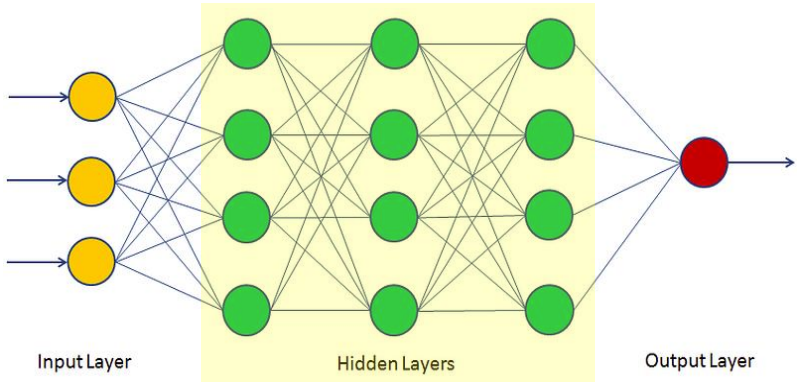




+) Activation Function

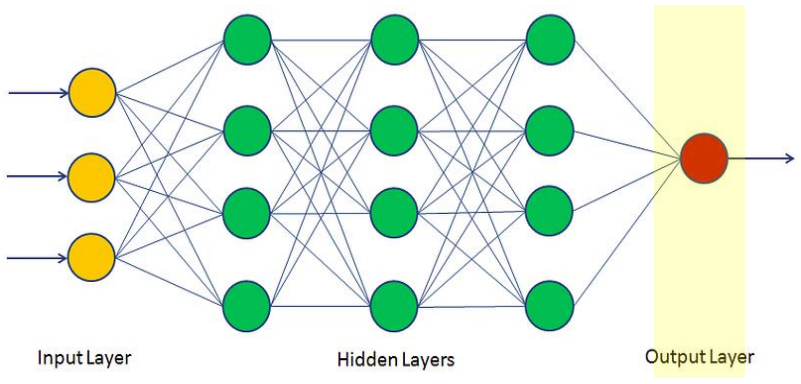
Activation Function을 어떻게 선택하는거죠?

Hidden Layer에 속하는 Activation Function의 관점



Hidden Layer의 Activation Function은
Output Layer로부터 넘어오는 Loss를 최대한
Input Layer까지 잘 전달할 수 있는 비선형 함수를 선택

Output Layer에 속하는 Activation Function의 관점



Output Layer의 Activation Function은
내가 원하는 출력을 뽑아내기 위해 가장 어울리는 함수를 선택

분류 – Sigmoid / Softmax
값 예측 – Linear
특별한 case) 작은 곤충의 길이 예측 ReLU (음수가 없음)

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

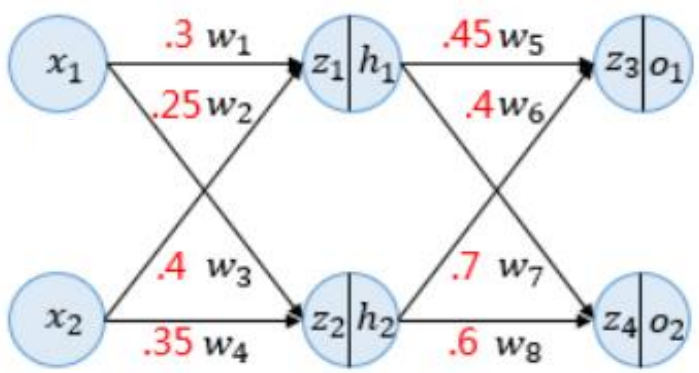
2. Error Calculation

현재 모델이 출력을 만들어냈으면, 그에 따른 Error를 계산하고 이를 바탕으로 가중치 갱신을 진행

	키 (x1)	몸무게 (x2)	o1의 정답	o2의 정답
1번 데이터	0.1	0.2	0.4	0.6

0.1

0.2



0.60944600 1번 Error ⟷ 0.4

0.66384491 2번 Error ⟷ 0.6

$$E = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2$$

Mean Squared Error

오차를 제곱한 값의 평균

$$E_{o1} = \frac{1}{2}(target_{o1} - output_{o1})^2 = 0.02193381$$

$$E_{o2} = \frac{1}{2}(target_{o2} - output_{o2})^2 = 0.00203809$$

$$E_{total} = E_{o1} + E_{o2} = 0.02397190$$

데이터는 분명 1개이긴 한데,
½가 붙어 있는 이유는
나중에 미분 연산이 필요한데,
제곱을 미분할 때 2가 곱해지니까,
2 * ½ = 1로 연산을 용이하게 하기 위함

+) Loss Function

Loss는 뭐고, Loss Function은 무엇이지?

$$L = \frac{1}{N}\sum_{i} L_i(f(x_i, W), y_i)$$

L	전체 **Loss**
i	데이터 **index** $(i = 1, 2, \ldots, N)$
x_i	i번째 입력 데이터
y_i	i번째 **Ground Truth**
W	모델의 가중치
$f(x_i, W)$	모델 (가중치를 통한 계산)
L_i	i번째 입력 데이터에 의한 **Loss**

Loss

모델의 Output과 실제 Ground Truth간의 차이를 계산해 전체 데이터에 대해 모두 더한 것

Loss Function

모델이 실제 정답과 비슷하게 출력을 만들지 못하는 정도를 측정하는 특별한 함수

Ex) 절댓값 차이, 제곱값 차이, Cross-Entropy, KL Divergence 등

+) Loss Function의 종류

Mean Absolute Error (평균 절대 오차)

$$MAE = \frac{1}{N}\sum_{i=1}^{n}\left|\hat{y}_i - y_i\right|$$

예측값과 Ground Truth 값의 차이에 절댓값을 씌운 것
주로 회귀 문제에서 사용되며, 절댓값을 씌운 것이라 이상치에 크게 민감하지 않음

Mean Squared Error (평균 제곱 오차)

$$MSE = \frac{1}{N}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

예측값과 Ground Truth값의 차이에 제곱을 씌운 것
주로 회귀 문제에서 사용되며, 제곱을 하기에 차이가 크면 클수록 훨씬 더 영향을 많이 줌 -> 이상치에 민감

위와 같이 회귀에 주로 사용되는 Loss Function은 다소 직관적으로 구성되어 있음
"너 정답이랑 차이가 있네? 그 차이가 곧 Penalty야"

+) Loss Function의 종류

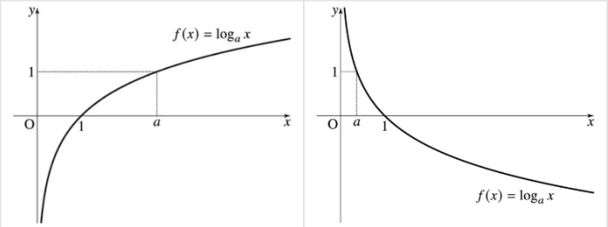
Binary Cross-Entropy Loss

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

예측값이 확률인 경우에 사용하는 Loss로,
이진 분류인 경우 2개 class에 대한 확률이 나오는데, 각 class별로 잘못한 만큼 penalty를 가짐

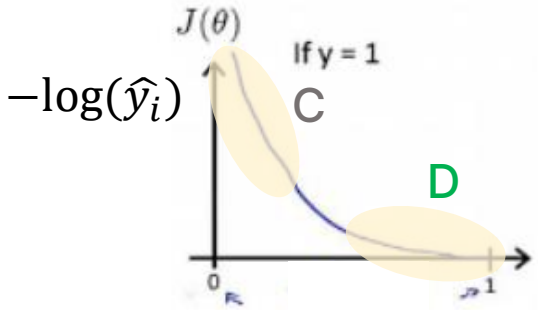
y_i = Ground Truth (0 / 1)
$\hat{y}_i$ = 모델의 예측 (0 ~ 1 확률)

Case A) 정답이 0일 때, 0에 가깝게 말해서 맞춘 경우 -> loss가 작아야 함
Case B) 정답이 0일 때, 1에 가깝게 말해서 틀린 경우 -> loss가 커야 함
Case C) 정답이 1일 때, 0에 가깝게 말해서 틀린 경우 -> loss가 커야 함
Case D) 정답이 1일 때, 1에 가깝게 말해서 틀린 경우 -> loss가 작아야 함



$a > 1$일 때 그래프 개형 | $0 < a < 1$일 때 그래프 개형

정답이 0인데, 내 모델이 0에 가깝게 말했을지언정 딱 0으로 예측한 것이 아니니까 loss는 발생해야 함.
하지만 과실을 따져봤을 때 loss의 비중이 달라져야한다는 것

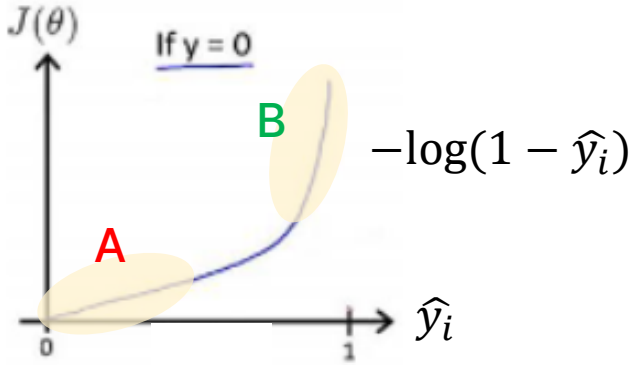


데이터 하나의 Loss

$$[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

정답이 0일 때 $(y_i = 0)$

$$[\cancel{y_i \log(\hat{y}_i)}^{\,0} + (1-\cancel{y_i}^{\,0})\log(1-\hat{y}_i)] \longrightarrow \log(1-\hat{y}_i) \longrightarrow$$



+) Loss Function의 종류

Multinomial Cross-Entropy

$$\frac{1}{N}\sum_{n=1}^{N}\sum_{i=1}^{K}-y_i^{(n)}\log a_i^{(n)}$$

앞서 확인한 Binary Cross-Entropy가 CE의 특별한 Case이고, CE가 일반화된 버전임
이 식 또한, 정답 label에 대해 확률을 적게 말하면 loss를 크게 주고, 확률을 높게 말하면 loss를 적게 주는 것

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.4 \\ 0.2 \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{aligned}\sum_{i=1}^{K}-y_i\log a_i &= -0\cdot\log(0.1) - 1\cdot\log(0.4) \\ &\quad -0\cdot\log(0.2) - 0\cdot\log(0.3) \\ &= -\log(0.4)\end{aligned}$$

Other Loss Function

Focal Loss

- Focal Loss는 객체 탐지 문제에서 클래스 불균형을 다루기 위해 제안되었습니다.
- Cross-Entropy Loss에 가중치를 부여하여, 잘못 분류된 예제에 더 집중하도록 합니다.

$$FL(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t)$$

Kullback-Leibler Divergence

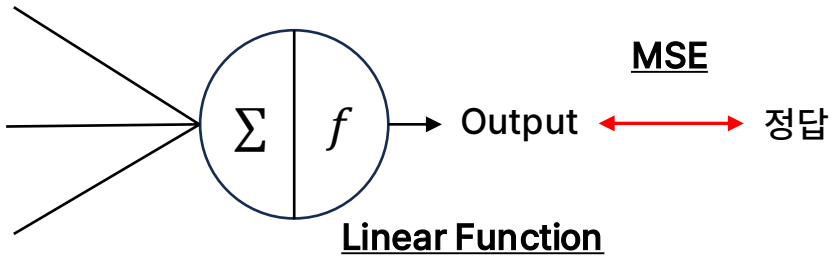
- KL Divergence는 두 확률 분포 사이의 차이를 측정하는 데 사용됩니다.
- 머신러닝에서는 주로 Variational Autoencoder (VAE)나 Generative Adversarial Network (GAN)에서 사용됩니다.

$$D_{KL}(P\|Q) = \sum_{x\in X}P(x)\log\left(\frac{P(x)}{Q(x)}\right)$$

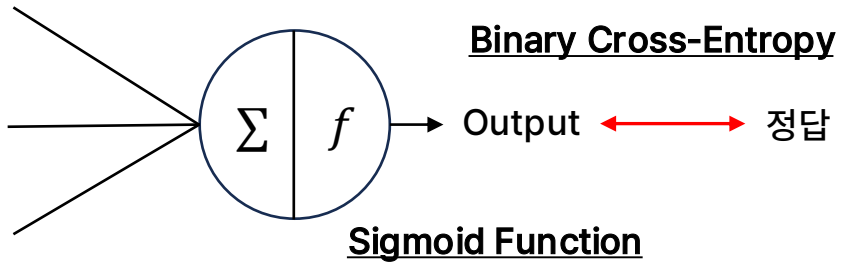
+) Loss Function – Activation Function 짝

앞서 다룬 Loss Function과 Output Layer의 Activation Function은 사실 상황에 따라 어울리는 짝이 있음

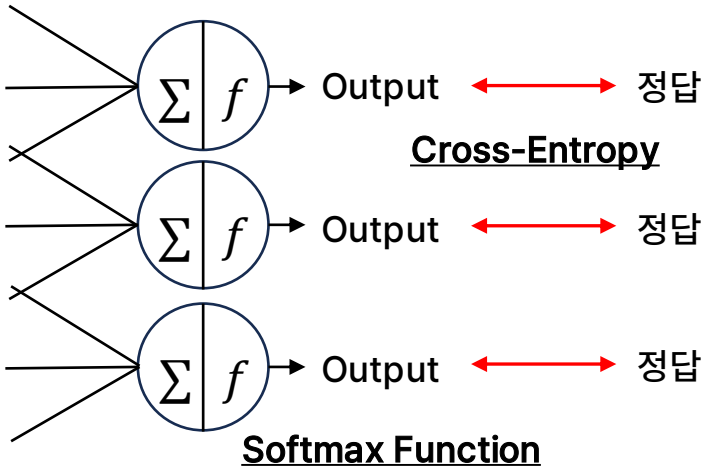
1. 특정한 수치를 예측하는 경우 (Regression)



2. 2개의 클래스를 예측하는 경우 (Binary Classification)



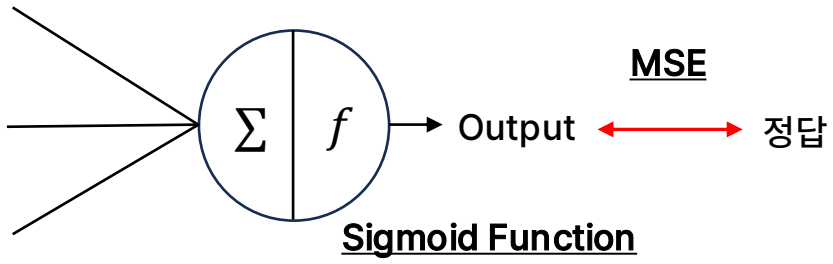
3. 여러 클래스를 예측하는 경우 (Classification)



+) Loss Function – Activation Function 짝

근데 내가 만약 Regression을 하긴 하는데, 곤충의 크기라서 범위가 0.3 ~ 0.7 사이에만 GT가 있다면?

Linear를 써야하나..? Sigmoid가 0 ~ 1 범위라 Sigmoid가 더 좋을 것 같은데..?



Sigmoid Function을 커스터마이징해서,
0.3 ~ 0.7 범위만 나오게 설정하고 MSE loss를 추가하는 방법

output=0.3+0.4·sigmoid(z)

결국 모든 Activation Function을 자세히 알아야, 원하는 Task/상황에 맞게 모델을 설계할 수 있음

3. BackPropagation – step 1

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

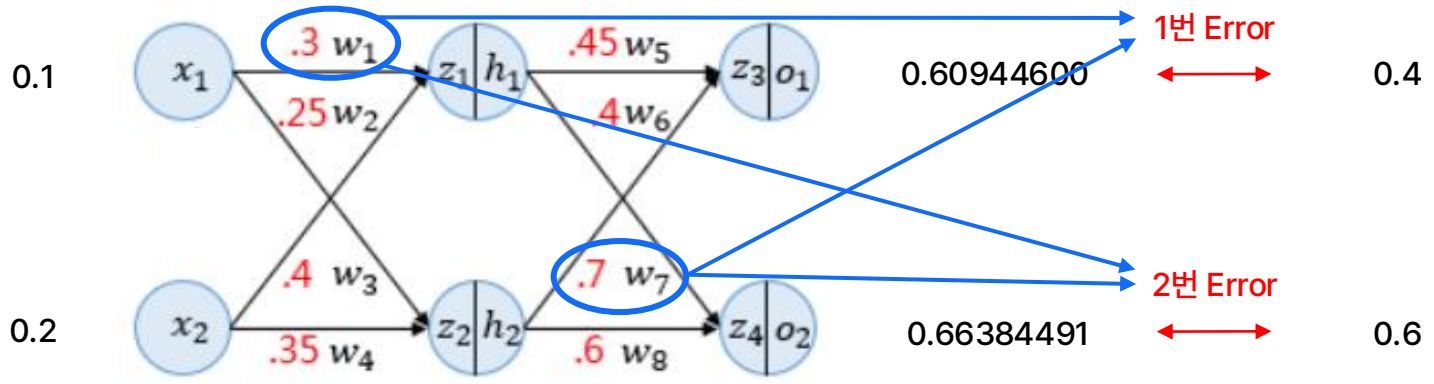
현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

Error를 계산했으면, 모든 가중치에 대해 Error에 미친 영향(Error를 줄이는 방향)을 찾아내야함



근데 가중치가 너무 많고, 얘네들끼리 너무 얽혀 있는데 어떻게 찾아내지..?

3. BackPropagation – step 1

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

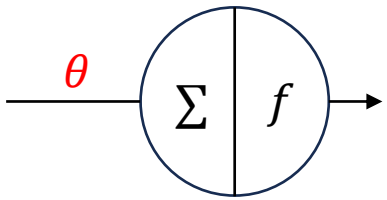
현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

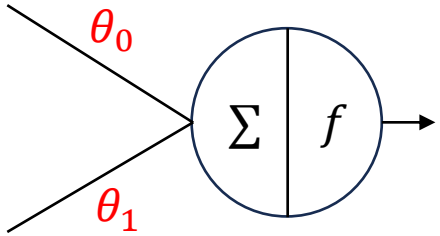
계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

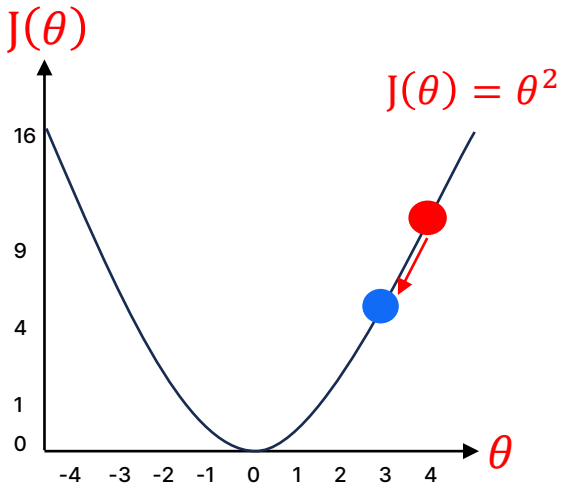
Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

이 말이 이해가 잘 안 될 수 있어서, 가중치가 1개 / 2개인 예시를 가져옴

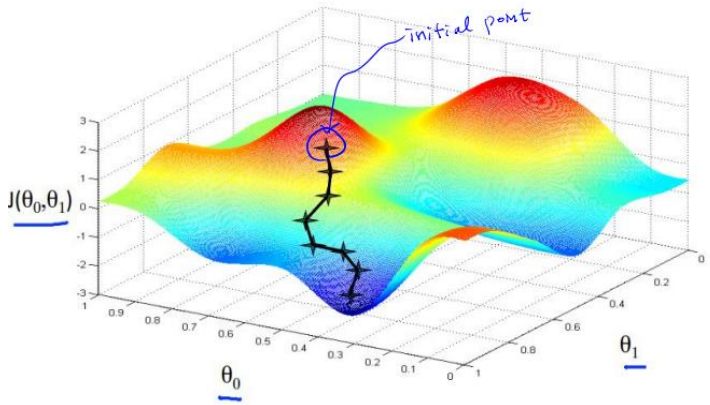


가중치가 1개인 경우
-> 최종 Output에 영향을 주는 가중치가 1개임
-> Loss Function이 가중치 1개에 대한 식으로 작성됨
-> Loss Function을 줄이고 싶으면 가중치 1개에 의한 영향(방향)만 찾으면 됨
-> 방향을 찾는 방법은 비용함수 식을 가중치 1개로 미분 하는 것





$$J(\theta_0, \theta_1) = 12\theta_0 + 8\theta_1$$



가중치가 2개인 경우
-> 최종 Output에 영향을 주는 가중치가 2개임
-> Loss Function이 가중치 2개에 대한 식으로 작성됨
-> Loss Function을 줄이고 싶으면 가중치 2개 각각에 의한 영향(방향)을 찾으면 됨
-> 방향을 찾는 방법은 비용함수 식을 가중치 2개 각각으로 미분 하는 것

3. BackPropagation – step 1

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

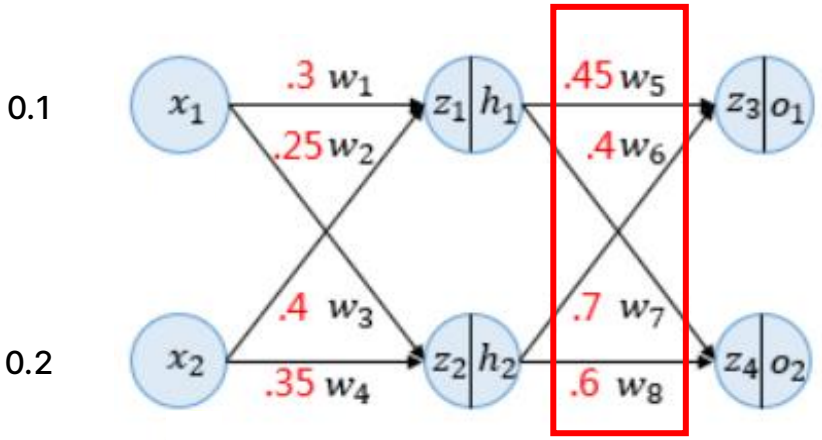
현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

우선, Hidden Layer가 있으니까 앞 뒤 가중치를 떼서 간단한 녀석들 먼저 생각해봅시다

Step 1에서 방향을 찾아야 할 가중치는 총 4개 입니다. w5, w6, w7, w8



0.60944600 ← 1번 Error → 0.4

0.66384491 ← 2번 Error → 0.6

이 상황에서 Loss Function을 정의했으니까, 각 가중치마다 Loss에 미치는 영향이 존재
따라서, Loss Function을 각 가중치로 미분하면 -> 각 가중치 별 Loss를 증가시키는 방향을 찾을 수 있음
그럼 그 방향에 (-) 부호를 붙이면 결과적으로 Loss를 줄이는 방향을 각 가중치별로 찾게 되는 것임

즉 $J(\theta)$를 w5로 미분한 값 / w6로 미분한 값 / w7으로 미분한 값 / w8으로 미분한 값을 찾아야 함

3. BackPropagation – step 1

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

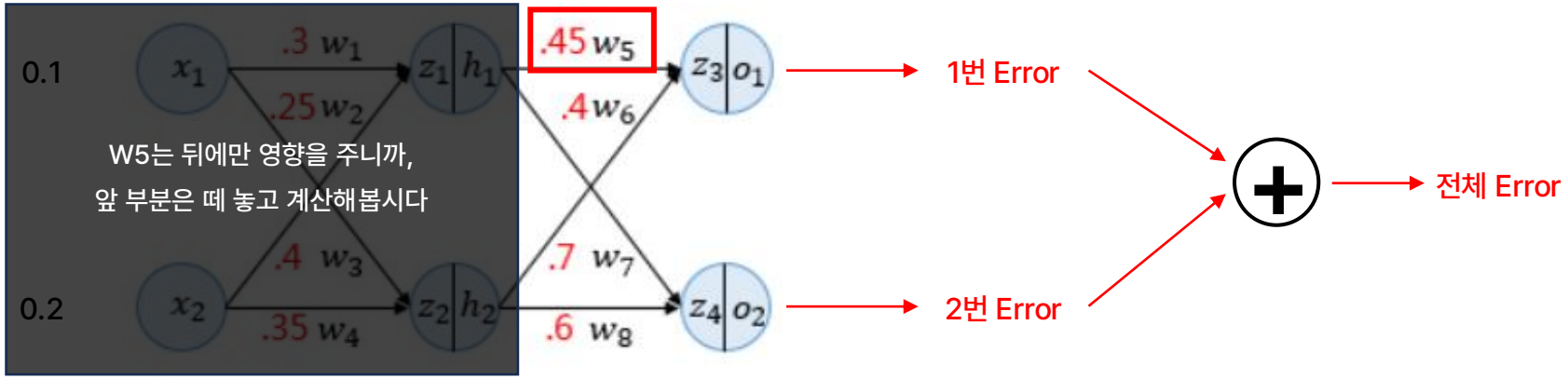
현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

Error를 w5로 미분한 값을 찾기 위해서, Error가 어떻게 계산되는지 확인해봅시다



$$E_{total}(o_1, o_2) = MSE(o_1) + MSE(o_2) = \frac{1}{2}(GT - o_1)^2 + \frac{1}{2}(GT - o_2)^2$$

$$o_1 = sigmoid(z_3) \qquad o_2 = sigmoid(z_4)$$

$$z_3 = h_1 * w_5 + h_2 * w_6 \qquad z_4 = h_2 * w_8 + h_1 * w_7$$

$$E = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2$$

핵심은 Error에 w5가 미치는 영향을 한번에 찾을 수 없다는 것
w5 -> z3 -> o1 -> Error 이렇게 연결되어 있음

3. BackPropagation – step 1

$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

이제 진짜 Error를 w5로 미분한 값을 찾아봅시다

$$E_{total}(o_1, o_2) = MSE(o_1) + MSE(o_2) = \frac{1}{2}(GT - o_1)^2 + \frac{1}{2}(GT - o_2)^2$$

$$o_1 = sigmoid(z_3) \qquad o_2 = sigmoid(z_4)$$

$$z_3 = h_1 * w_5 + h_2 * w_6 \qquad z_4 = h_2 * w_8 + h_1 * w_7$$

근데 Error에 w5가 바로 연결된게 아니니까, Chain Rule에 따라 풀어서 계산해야 함

$$\frac{\partial E_{total}}{\partial w_5} = \ ?$$

Sigmoid 함수의 미분

$$f(x) = \frac{1}{1+e^{-x}}$$

$$\frac{df}{dt} = f(x) \cdot (1 - f(x))$$

$$\boxed{\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial w_5}}$$

$$\frac{\partial E_{total}}{\partial o_1} = 2 \times \frac{1}{2}(target_{o1} - output_{o1})^{2-1} \times (-1) + 0 \ = -(target_{o1} - output_{o1}) = -(0.4 - 0.60944600) = 0.20944600$$

$$\frac{\partial o_1}{\partial z_3} = o_1 \times (1 - o_1) = 0.60944600(1 - 0.60944600) = 0.23802157$$

$$\frac{\partial z_3}{\partial w_5} = h_1 = 0.51998934$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.20944600 \times 0.23802157 \times 0.51998934 = 0.02592286$$

이 기울기의 반대로(-0.02592286) w5를 수정하면 Error가 감소함

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

3. BackPropagation – step 1

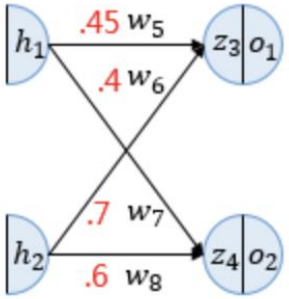
이제 찾아낸 방향을 바탕으로 가중치를 갱신해봅시다

$$new\ \theta = current\ \theta - \alpha \underline{\nabla J(\theta)}$$

Error를 가중치로 미분한 것

$$w_5^+ = w_5 - \alpha \frac{\partial E_{total}}{\partial w_5} = 0.45 - 0.5 \times 0.02592286 = 0.43703857$$

원래 가중치



$$\frac{\partial E_{total}}{\partial w_6} = \frac{\partial E_{total}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial w_6} \rightarrow w_6^+ = 0.38685205$$

$$\frac{\partial E_{total}}{\partial w_7} = \frac{\partial E_{total}}{\partial o_2} \times \frac{\partial o_2}{\partial z_4} \times \frac{\partial z_4}{\partial w_7} \rightarrow w_7^+ = 0.69629578$$

$$\frac{\partial E_{total}}{\partial w_8} = \frac{\partial E_{total}}{\partial o_2} \times \frac{\partial o_2}{\partial z_4} \times \frac{\partial z_4}{\partial w_8} \rightarrow w_8^+ = 0.59624247$$

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

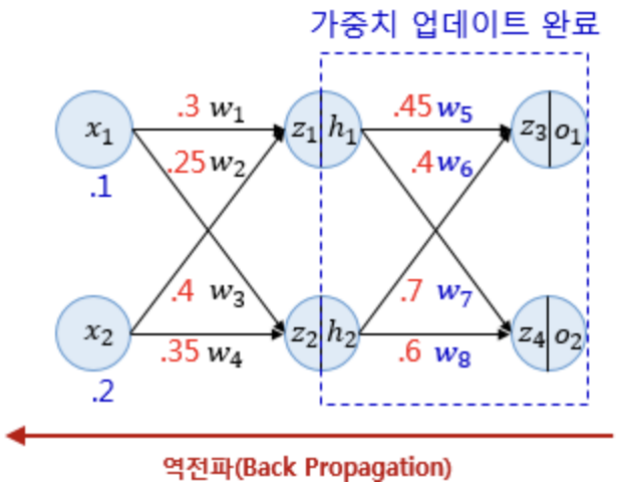
현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

3. BackPropagation – step 2

이제 뒷 부분은 수정이 완료되었으니까, 앞 부분으로 가봅시다



Step 2에서 방향을 찾아야 할 가중치는 총 4개 입니다. w1, w2, w3, w4

근데 step1보다 조금 더 어려울 것 같은게, 아까는 output node랑 바로 연결되어 있었는데 이번에는 중간에 hidden layer가 존재함

즉 $J(\theta)$를 w1로 미분한 값 / w2로 미분한 값 / w3로 미분한 값 / w4로 미분한 값을 찾아야 함

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

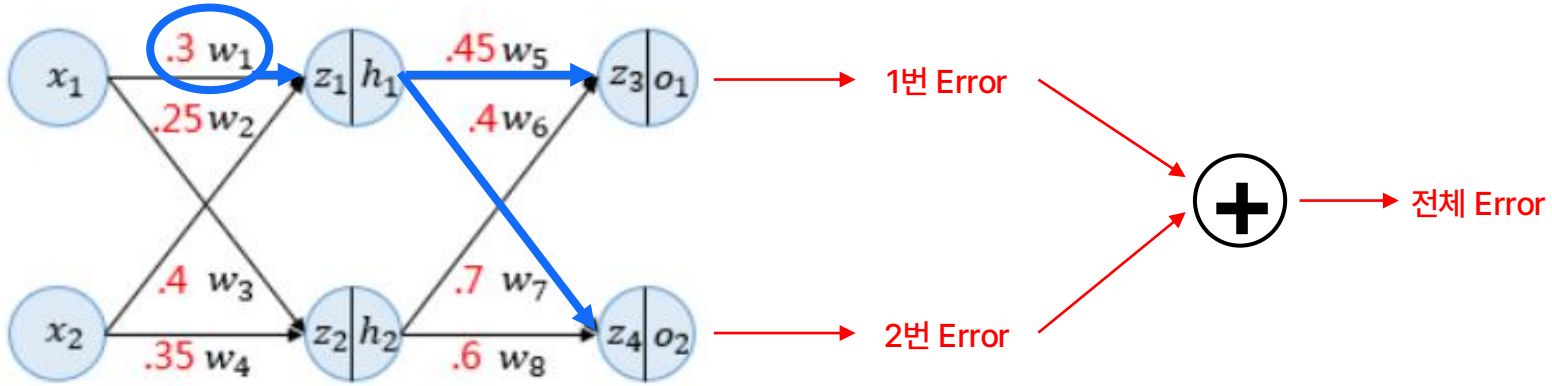
현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

3. BackPropagation – step 2

이번에는 w1을 예시로 Error를 줄이는 방향을 계산해 봅시다



$$\frac{\partial E_{total}}{\partial w_1} = \boxed{\frac{\partial E_{total}}{\partial h_1}} \times \frac{\partial h_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

$$\boxed{\frac{\partial E_{total}}{\partial h_1} = \boxed{\frac{\partial E_{o1}}{\partial h_1}} + \boxed{\frac{\partial E_{o2}}{\partial h_1}}}$$

h1은 Error 1에도 영향을 주고, Error 2에도 영향을 줌

$$\frac{\partial E_{o1}}{\partial h_1} = \frac{\partial E_{o1}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial h_1} \qquad \frac{\partial E_{o2}}{\partial h_1} = \frac{\partial E_{o2}}{\partial o_2} \times \frac{\partial o_2}{\partial z_4} \times \frac{\partial z_4}{\partial h_1}$$

현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

3. BackPropagation – step 2

모든 가중치에 대해 기울기 방향을 찾고, 갱신을 진행함

$$w_1^+ = w_1 - \alpha \frac{\partial E_{total}}{\partial w_1} = 0.3 - 0.5 \times 0.00080888 = 0.29959556$$

$$\frac{\partial E_{total}}{\partial w_2} = \frac{\partial E_{total}}{\partial h_1} \times \frac{\partial h_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_2} \rightarrow w_2^+ = 0.24919112$$

$$\frac{\partial E_{total}}{\partial w_3} = \frac{\partial E_{total}}{\partial h_2} \times \frac{\partial h_2}{\partial z_2} \times \frac{\partial z_2}{\partial w_3} \rightarrow w_3^+ = 0.39964496$$

$$\frac{\partial E_{total}}{\partial w_4} = \frac{\partial E_{total}}{\partial h_2} \times \frac{\partial h_2}{\partial z_2} \times \frac{\partial z_2}{\partial w_4} \rightarrow w_4^+ = 0.34928991$$

4. Checking

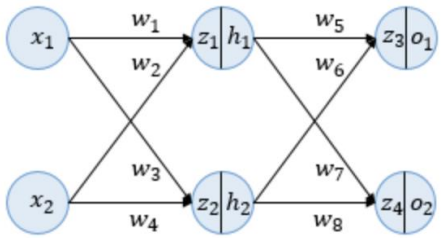
현재 모델에 특정 데이터를 넣고 Output을 만들어낸다

현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다

계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다

Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

새롭게 갱신된 가중치가 실제로 Error를 줄이는지 확인해야 함



$$z_1 = w_1x_1 + w_2x_2 = 0.29959556 \times 0.1 + 0.24919112 \times 0.2 = 0.07979778$$

$$z_2 = w_3x_1 + w_4x_2 = 0.39964496 \times 0.1 + 0.34928991 \times 0.2 = 0.10982248$$

$$h_1 = sigmoid(z_1) = 0.51993887$$

$$h_2 = sigmoid(z_2) = 0.52742806$$

$$z_3 = w_5h_1 + w_6h_2 = 0.43703857 \times h_1 + 0.38685205 \times h_2 = 0.43126996$$

$$z_4 = w_7h_1 + w_8h_2 = 0.69629578 \times h_1 + 0.59624247 \times h_2 = 0.67650625$$

$$o_1 = sigmoid(z_3) = 0.60617688$$

$$o_2 = sigmoid(z_4) = 0.66295848$$

$$E_{o1} = \frac{1}{2}(target_{o1} - output_{o1})^2 = 0.02125445$$

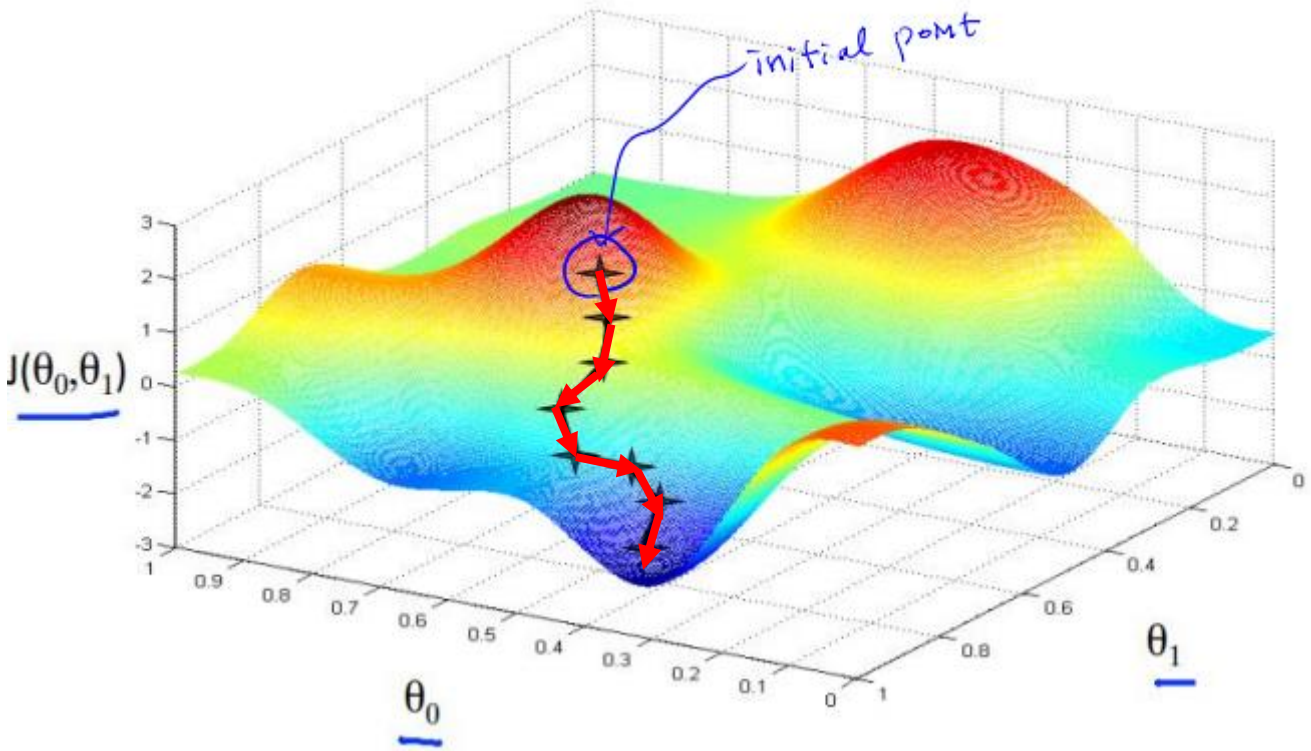
$$E_{o2} = \frac{1}{2}(target_{o2} - output_{o2})^2 = 0.00198189$$

$$E_{total} = E_{o1} + E_{o2} = 0.02323634 \longleftarrow E_{total} = E_{o1} + E_{o2} = 0.02397190$$

실제로 감소함

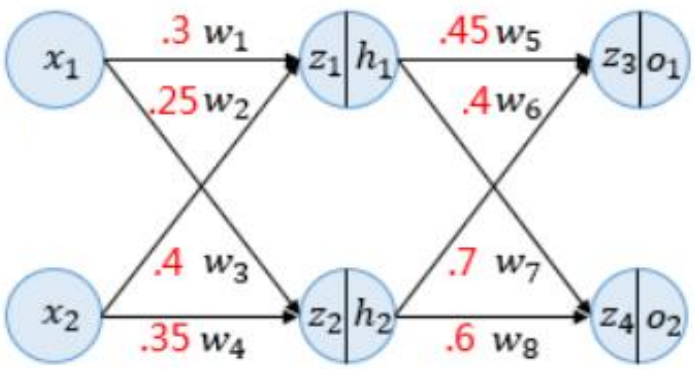
Gradient Descent의 핵심

결국 Gradient Descent는 엄청 깊은 층을 가지는 상황 속에서,
각 가중치가 Error에 어떤 영향을 미치는지 체크하고, 줄일 수 있는 방향을 계산하여
그 방향으로 조금 조금식 이동하는 것



Gradient Descent의 핵심

근데, 그 방향을 찾아내기 위해선 얽히고 얽힌 그 과정을 모두 미분해야 하는 것임



Output Layer와 연결된 가중치에 필요한 계산

$$w_5 \rightarrow z_3 \rightarrow o_1 \rightarrow E_{total}$$

$$\boxed{\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial w_5}}$$

1층 Hidden Layer에 연결된 가중치에 필요한 계산

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_5 \rightarrow z_3 \rightarrow o_1 \searrow$$

$$E_{total}$$

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_7 \rightarrow z_4 \rightarrow o_2 \nearrow$$

$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial o_1} \times \frac{\partial o_1}{\partial z_3} \times \frac{\partial z_3}{\partial h_1} + \frac{\partial E_{o2}}{\partial o_2} \times \frac{\partial o_2}{\partial z_4} \times \frac{\partial z_4}{\partial h_1}\right) \times \frac{\partial h_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1}$$

Gradient Descent의 핵심

Output Layer와 연결된 가중치에 필요한 계산

$$w_5 \rightarrow z_3 \rightarrow o_1 \rightarrow E_{total}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial o_1} \times \boxed{\frac{\partial o_1}{\partial z_3}} \times \frac{\partial z_3}{\partial w_5}$$

1층 Hidden Layer에 연결된 가중치에 필요한 계산

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_5 \rightarrow z_3 \rightarrow o_1 \rightarrow E_{total}$$

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_7 \rightarrow z_4 \rightarrow o_2 \rightarrow E_{total}$$

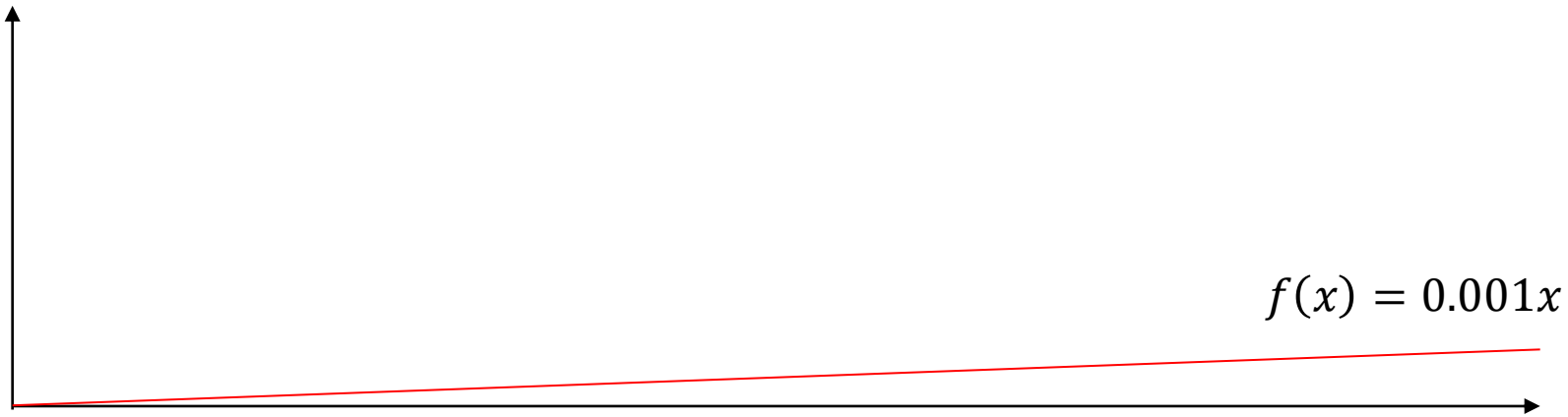
$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial o_1} \times \boxed{\frac{\partial o_1}{\partial z_3}} \times \frac{\partial z_3}{\partial h_1} + \frac{\partial E_{o2}}{\partial o_2} \times \boxed{\frac{\partial o_2}{\partial z_4}} \times \frac{\partial z_4}{\partial h_1} \right) \times \boxed{\frac{\partial h_1}{\partial z_1}} \times \frac{\partial z_1}{\partial w_1}$$

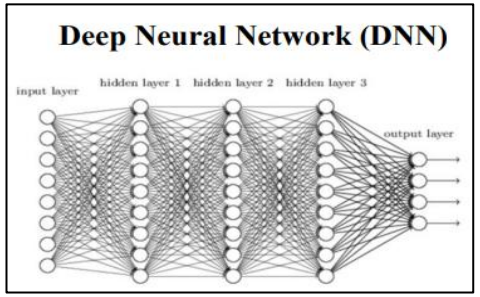
빨간 box는 activation function의 미분임

층이 쌓이면 쌓일수록 가중치를 갱신하기 위해 activation function의 미분이 여러번 곱해지는 것임

Gradient Descent의 핵심

근데, activation function이 전 구간에서 기울기가 0.001이라면 ?



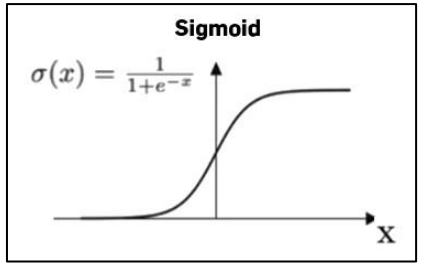


$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial o_1} \times \overset{0.001}{\boxed{\frac{\partial o_1}{\partial z_3}}} \times \frac{\partial z_3}{\partial h_1} + \frac{\partial E_{o2}}{\partial o_2} \times \overset{0.001}{\boxed{\frac{\partial o_2}{\partial z_4}}} \times \frac{\partial z_4}{\partial h_1}\right) \times \underset{0.001}{\boxed{\frac{\partial h_1}{\partial z_1}}} \times \frac{\partial z_1}{\partial w_1} = \underline{0.00001(\text{수치})}$$

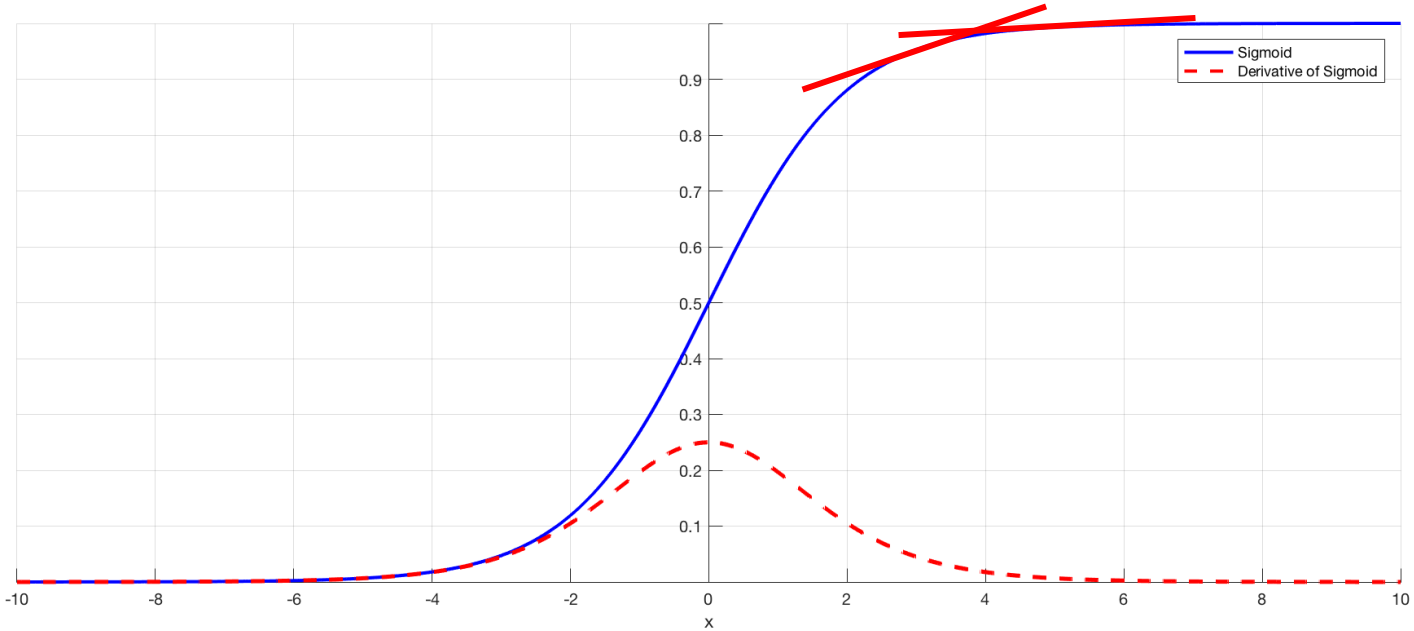
이만큼 수정한다고 모델의 출력이 달라질까요?

즉, Hidden Layer가 많으면 많아질수록 activation function에 따라 가중치 갱신이 어려워짐 !!!

+) Activation Function



Sigmoid Function

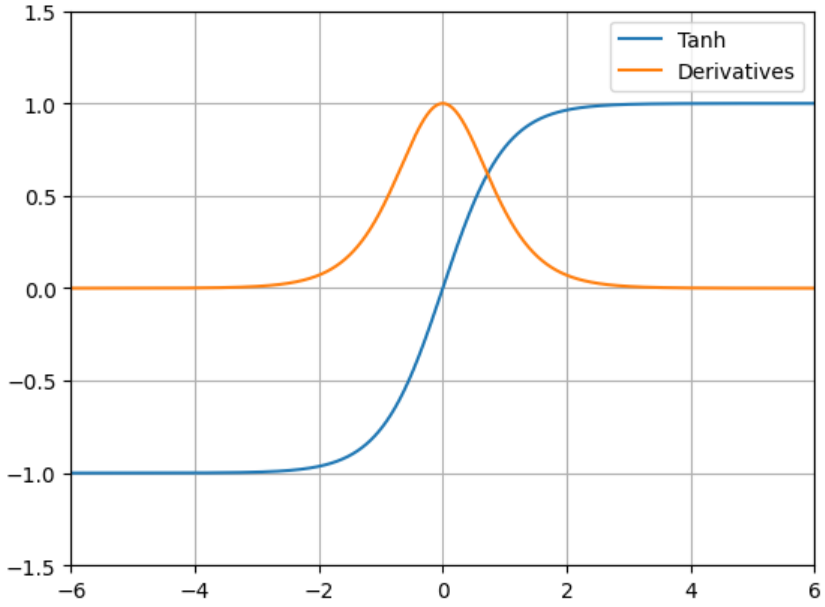


Output Node에서 Classification을 할 때를 제외하고 Hidden Layer의 Activation Function으로는 주로 사용하지 않음

거의 대부분의 구간에서 기울기 값이 0에 수렴하기 때문에, Gradient를 넘겨주기 적절하지 못함. (가중치 갱신 어려움)
또한, exp() 식이 포함되어 있기 때문에 미분을 계산하기가 다소 복잡함 (Computational Cost 높음)

+) Activation Function

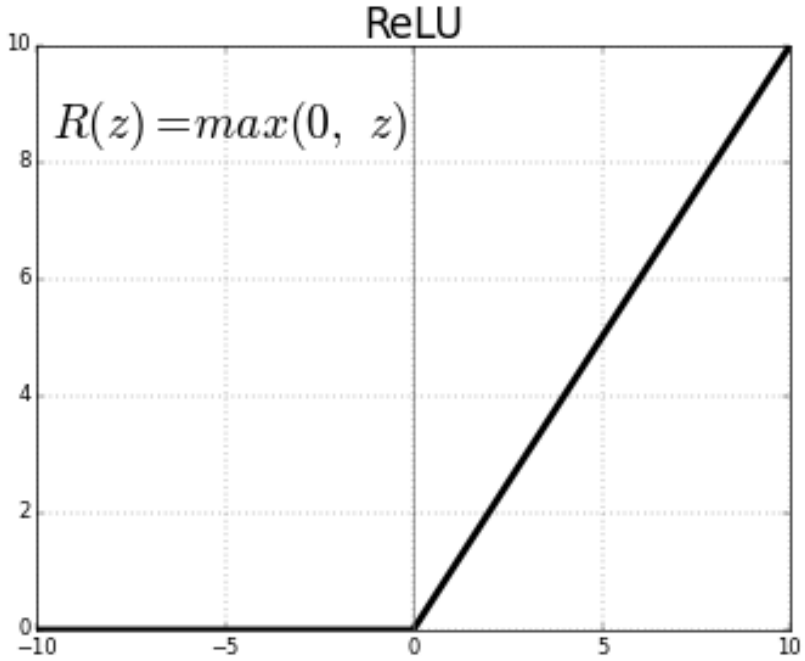
tanh Function



Sigmoid의 문제를 해결하고자 등장했으나, 여전히 대부분의 구간에서 미분 값이 0에 가까워 Gradient 소실 문제가 발생함

+) Activation Function

ReLU Function

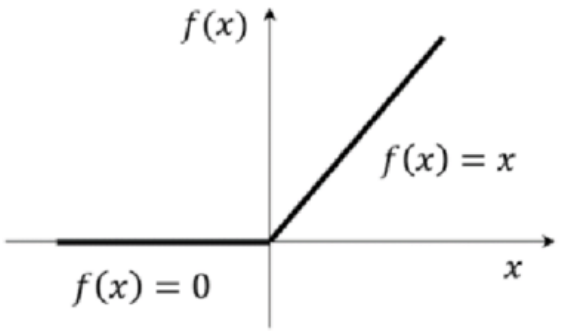


양수에서는 항상 기울기를 1로, 음수에서는 0으로 계산하는 ReLU가 등장함
Sigmoid, tanh 들의 Gradient 소실 문제를 하였으며, 계산도 쉬워서 학습 속도 및 수렴 속도도 매우 빨라짐

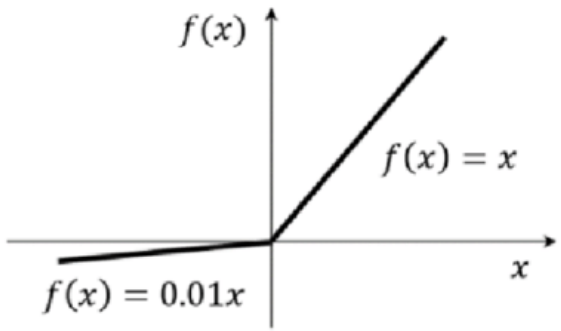
물론, 음수에서는 0이니까 기울기를 못 전달하는거 아닌가요? 라고 말할 수 있으나
애초에 Gradient를 잘 전달해서 음수가 되는 경우가 거의 발생하지 않음 / 또한 0 이하에서 일부를 죽이기 때문에 오히려 Overfitting 완화

+) Activation Function

Leaky ReLU Function



ReLU activation function



LeakyReLU activation function

이를 개선하고자 Leaky ReLU 함수 등이 등장했고, 여기서부턴 휴리스틱하게 자기 모델에 맞는거 선택하면 됨

Transformer에는 GELU라는 함수가 좋다고 하기는 함